

1. 绪论

题型：选择10；判断10，简答6

分布式系统

- **定义**：分布式系统是若干独立计算机的集合，这些计算机对用户来说就像是一个单机的系统

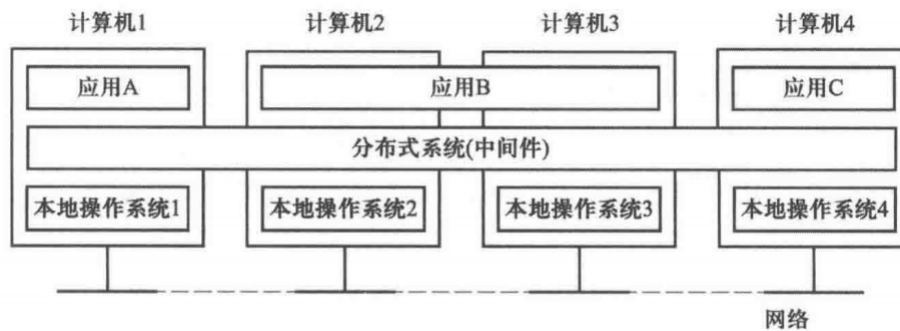
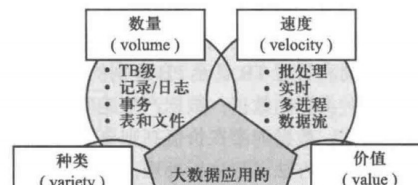


图 1.1 分布式系统^[1]

- 分类
 - 根据依赖硬件的不同：基于计算机构建（计算系统、信息系统） VS 基于微型设备构建
 - 根据用途的不同：分布式计算系统（集群/网格计算、云计算等） VS 分布式信息系统（事务处理、企业应用集成等）
- 数据管理系统发展历史
 - 1：层次、网状数据库系统：可视作数据组织与存取的工具，将数据的存储和访问功能从应用程序中分离
 - 2：关系数据库系统：OLTP：联机事务处理（事务处理、查询优化、数据访问）
 - 3：数据仓库系统：OLAP：联机分析处理（复杂查询）
 - 4：大数据管理系统：分布式可扩展、异地多备份高可用性架构、多数据模型支持、多应用负载类型支持
- 大数据背景下数据管理面临的挑战
 - 5V：
 - volume：数量
 - variety：种类：结构化/非结构化数据
 - velocity：速度



- variety: 种类: 结构化/非结构化数据
- velocity: 速度
- value: 价值
- veracity: 质量

○ 为数据管理系统带来的机遇和挑战

- 数据量大
- 数据种类多
- 产生速度快
- 潜在价值高
- 数据质量低

- 面向数据管理的分布式系统

○ 数据存储——分布式存储系统: 不考虑数据模式

- 分布式文件系统HDFS
- 分布式消息存储系统Kafka
- 分布式缓冲区管理系统Alluxio

○ 数据组织与访问——分布式NoSQL系统: 考虑数据模式

- 包括kv、文档、图数据库
- BigTable、Dynamo、Cassandra、Neo4j等
- 仅支持部分事务管理, 不保证支持完整的ACID特性

○ 数据库的查询处理——分布式计算系统

- 批处理/流处理等执行引擎的处理模型
- MapReduce/Spark/Flink
- 仅借鉴查询处理, 不考虑事务管理

○ 数据库的事务管理——分布式NewSQL系统

- 支持满足ACID的事务管理
- Spanner、OceanBase、TiDB等

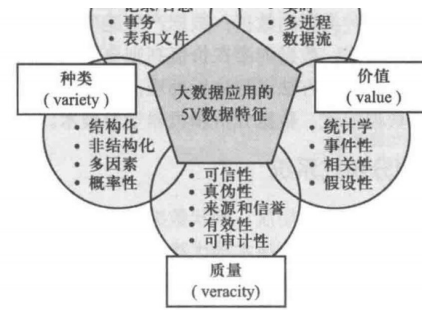
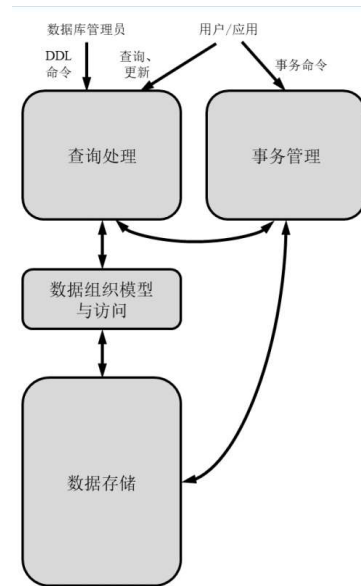


图 1.2 大数据应用的数据特征^[5]



分布式计算系统

- **定义**: 分布式计算系统是作用于**若干独立计算机**之上、使得这些计算机能够**协同执行计算**完成某项应用的软件系统

分布式计算系统归根结底是为了解决某些类别的应用问题而设计的分布式系统

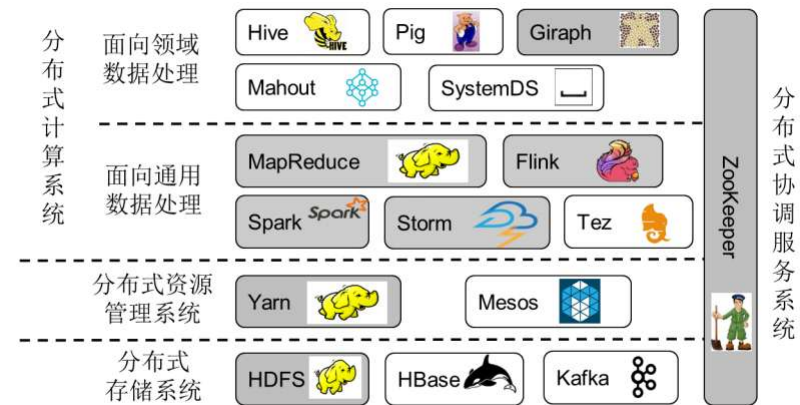
- 应用类型

○ 应用负载类型

- 计算密集型应用：CPU为首要限制
- **数据密集型应用**：IO带宽为首要限制（本书面向）

○ 应用驱动类型

- 通用数据处理：
 - 批处理系统（吞吐）：MapReduce、Spark、Tez
 - 流处理系统（实时）：storm、Spark Streaming
 - 批流处理系统：Flink、Structured Streaming
- 领域数据处理：
 - 图处理系统：Giraph、GraphX
 - 机器学习处理系统：Mahout、S伊斯特曼DS，
 - 查询分析系统：Hive、Pig



本书组织结构

- **设计思想**：该系统应对的应用场景如何？为什么需要构造这个系统？该系统所处理的数据模型是怎样的？该系统采用何种计算模型？如果系统支持迭代计算，则应采用何种迭代模型？
- **体系架构**：作为一个分布式系统，该系统包含哪些部件，即由哪些进程或线程构成？各个部件分别扮演怎样的角色？系统中的各个部件以何种次序共同工作才能保障用户应用程序的顺利执行？
- **工作原理**：系统如何根据用户编写的应用程序生成相应的执行计划，以及是否可以进行优化？面对数据密集型应用，系统如何减少数据IO代价（此处IO的代价表现为节点本地读写数据的磁盘IO代价，以及节点之间通过网络进行数据传输的代价，后者在分布式场景下显得尤为突出）
- **容错机制**：商业服务器集群中的节点发生故障是常见现象，系统如何保证某些节点存在故障的情况下不影响用户应用程序的正常执行，即故障对用户透明？早期的并行计算系统部署于高性能服务器甚至超级计算机中，通常无需考虑容错问题。容错机制是本书讨论的分布式计算系统与早期分布式计算系统之间存在的显著差异。
- **编程示例**：使用系统的编程接口进行简单编程的一般步骤是怎样的？编程过程是否存在通用的框架？如何提高某些程序的性能？使用不同系统实现同一个应用所编写的程序有何差异？

习题

- 1.简述分布式系统的概念

分布式系统是若干个独立计算机的集合，这些计算机对于用户来说就像是一个单机的系统。为了使用户体验上觉得这是一个单机的系统，分布式系统通过一个“软件层”组织而成，该软件层在逻辑上位于由应用组成的高层和于本地系统组成的低层之间，有时被称为“中间件”。

- 2.分布式系统可以分为哪些类型

- 硬件不同：基于计算机构建的分布式系统和基于微型设备构建的分布式系统
- 用途不同：基于计算机构建的分布式系统可进一步分为分布式计算系统、分布式信息系统。
 - 分布式计算系统智能高性能计算和大规模信息处理，一组计算机通过局域网构建的集群可以提供强大的计算能力。云计算的日益成熟让传统的网格计算退出了历史舞台。
 - 分布式信息系统主要利用分布式系统技术构建信息管理系统，包括用户实现事务处理、企业应用的集成等
 - 基于微型设备构建的分布式系统又可成为分布式普适系统，包括智慧家庭系统，电子健康系统，传感网格系统等。

- 3.数据管理系统的发展历程

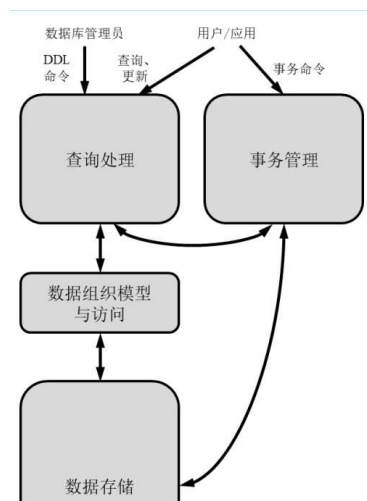
- 1.层次、网状数据库系统：侧重数据的组织与存取，将数据的存储和访问功能从应用中分离
- 2.关系数据库：OLTP，联机事务处理，事务处理、查询优化、数据访问等
- 3.数据仓库系统：OLAP联机分析处理，复杂查询
- 4.大数据管理系统：分布式、可扩展、高可用

- 4.简述大数据的特征

- 数量大
- 种类多
- 速度快
- 价值高
- 质量低

- 5.数据库系统包含哪些主要的功能模块

- 数据存储
- 数据组织与访问
- 查询处理



分布式系统

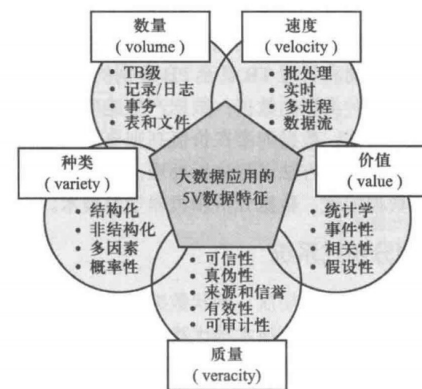
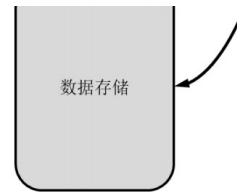


图 1.2 大数据应用的数据特征^[5]

- 数据组织与访问
- 查询处理
- 事务管理



- 6.数据密集型应用和计算密集型应用有何区别？

- 计算密集型应用处理时CPU性能成为瓶颈，数据密集型应用处理时IO带宽成为瓶颈。

- 7.通用数据处理系统可以分为哪些类别？分别适用于哪些场景？

- 批处理：静态数据，吞吐量大，实时性要求低
- 流计算：动态数据，吞吐量低，实时性要求高
- 批流融合：上述两种场景，也可用于同一场景下不同模块的数据特征和计算需求不同的场景。

- 8.分布式计算系统本身并无数据存储功能，则其数据从何而来，输出结果又走向何方？

数据来自于分布式存储系统，资源管理交给分布式资源管理系统，高可用性保证交给分布式协调服务系统。输出结果给程序员复用或者直接交给用户。

2. Hadoop文件系统

发展简史

- Hadoop文件系统HDFS是Hadoop项目的一部分，源于Lucene项目（Nutch），发起者：Doug Cutting
- Hadoop常被视为谷歌发表的学术论文的开源实现（HDFS-GFS Hadoop MapReducep-Google MapReduce）

设计思想

- 如何存储上百GB/TB级别的大文件？

文件分块存储：HDFS将大文件分块，块的大小设置为128MB或其他大小，这些文件块可以分布到集群中不同的节点上，因此，HDFS采用分而治之的策略，令多个节点对逻辑层面的大文件在物理节点进行分布式存储。

- 如何保证文件系统的容错？

分块冗余存储：HDFS将大文件切分成块，每个小块同时进行冗余备份，由于多个备份会分散到不同的节点上，因此除非存放某一文件块的所有节点均发生故障，否则HDFS在部分节点发生故障的情况下依然可以访问该文件块。

- 如何进行大文件的并发读写控制？

简化文件读写：由于HDFS的文件往往一次写入后不再修改而是多次读取，因此HDFS简化了文件读写模型，采用“一次写入、多次读取”的方式，即可避免读写冲突。此外，与操作系统中的文件系统需要支持文件的顺序写入以及随机写入不同，HDFS**仅需支持顺序写入而不支持随机写入**。

体系架构：主从架构

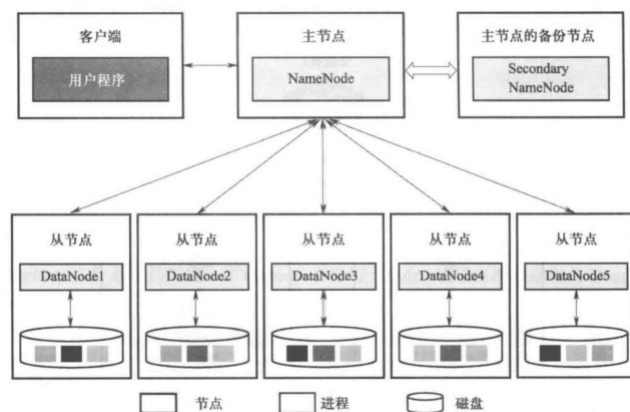


图 2.2 HDFS 架构图

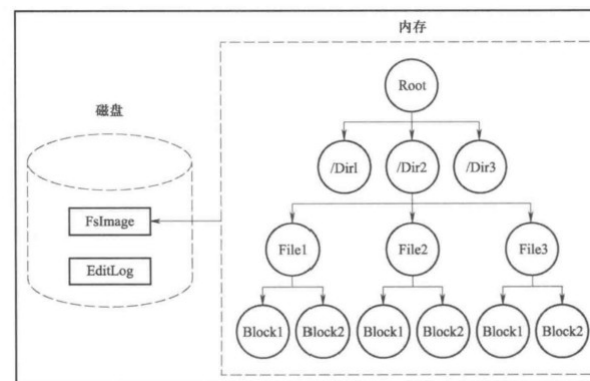


图 2.3 NameNode 数据结构

- 功能

- 主节点NameNode：负责HDFS的管理工作，包括管理文件目录结构、位置等元数据，维护DataNode的状态等，并不实际存储文件。
 - 主节点的备份节点Secondary NameNode：充当NameNode的备份，当NameNode发生故障时理由SecondaryNameNode进行恢复。
 - 从节点DataNode：负责存储文件，根据NameNode的控制信息存储和管理对应的文件块，并定期向NameNode汇报自身的状态。
- 主节点NameNode的数据结构（上右图）
- HDFS的目录结构与Linux文件系统的目录结构很相似
 - 块信息仅指示文件块存储的位置，并不实际存储文件。
 - HDFS将位于内存中的目录结构及其元数据写入磁盘的**FsImage**文件：HDFS目录结构的一个本地快照。
 - 针对目录及文件的修改操作，除了修改内存中的树形目录外，还以日志形式记录到磁盘的**EditLog**中。

- 备份主节点操作

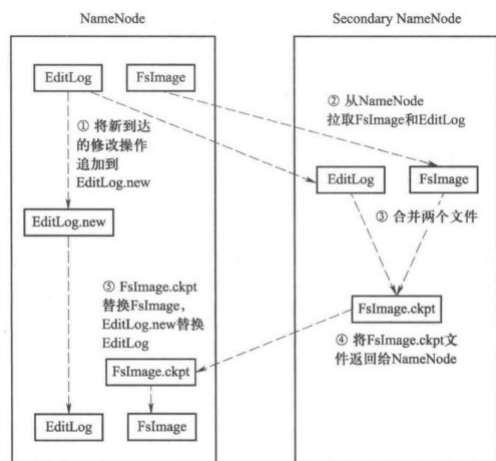


图 2.4 NameNode 与 Secondary NameNode

- DataNode数据节点
- 是负责数据块实际存储的节点，根据NameNode控制信息进行数据块的存储和检索。
 - 通常将HDFS部署在一组安装有Linux操作系统的服务器上，因此HDFS中的数据块以Linux文件的形式保存在DataNode的本地文件系统中。
- 应用程序执行流程
- 客户端向NameNode发起文件操作请求

- 如果是读写文件操作——NameNode告知客户端文件块存储的位置信息。
- 如果是创建/修改等目录或文件操作，NameNode成功修改文件目录结构后即结束该操作。
删除操作，HDFS不会立即删除DataNode上的数据块，而是等到特定时间才会真正删除。
- 对于读写文件操作，客户端获知具体位置信息后再与DataNode进行读写交互。

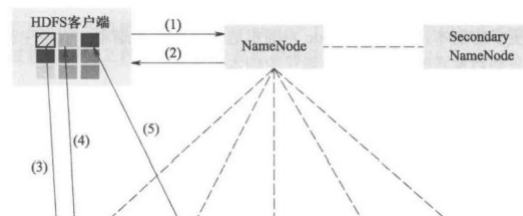
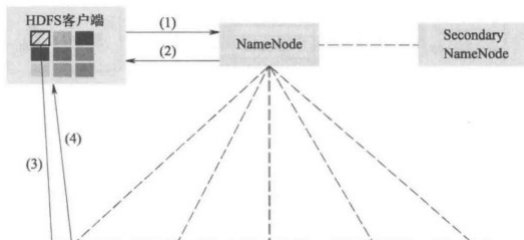
工作原理

- 文件分块与备份：启发式文件块存放策略

- 第1个副本——支持快速写入：若客户端与某一DataNode位于同一物理节点，则HDFS将第一个副本放置于该DataNode；若客户端和任一DataNode都不在一个物理节点，则挑选一个磁盘容量和CPU性能俱佳的节点
- 第2个副本——整体减少跨机架的网络流量：NameNode将第二个部分放置于不同于第一个副本所在机架的某一节点。
- 第3个副本——应对故障发生时的文件块读取：NameNode将第三个副本放置于第1个副本所在机架的不同节点。
- 更多副本：随机选择节点放置。

- 文件写入

- 客户端与NameNode通信，请求写入一个文件块。
- NameNode根据上面的启发策略决定文件块放置的DataNode，并将DataNode的位置告诉客户端
- 客户端与DataNode1建立联系，DN1与DN2建立联系，DN2与DN3建立联系，客户端将文件块以流水线的方式写入这些DN。
- DN3写入完毕后向DN2发送确认消息，DN2-DN1，DN1-客户端发送确认消息，表示该文件块成功写入。
- 单个文件块通常为64MB或者更大，客户端通常以更小例如64kb的数据包为单位传输，DN1收到一个数据包后，除了写入本地缓存，会立即转发给DN2，同理DN2收到后也会立即给DN3，因此一系列数据包的发送过程呈现为流水线的方式。当该文件块写入完毕后，客户端再次向NameNode发送通信，请求写入下一个文件块并重复上述过程——单个文件块以流水线方式传输，文件块之间以阻塞方式传输。



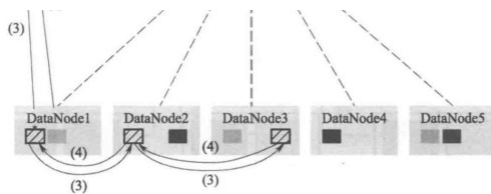


图 2.7 客户端向 HDFS 写入文件块

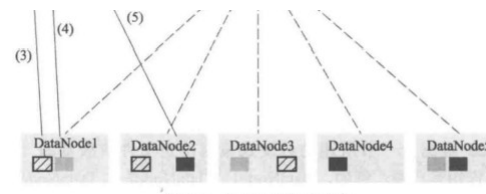


图 2.8 从 HDFS 读取文件

- 文件读取

- 客户端与NameNode通信，请求读取一个文件
- NameNode根据文件的路径等信息判断读取请求是否合法，如果合法则向客户端返回文件中所有数据块的存放地址
- 对于第一个数据库，客户端从距离最近的存放该数据块的DataNode中读取数据
- 第一个数据块读取完毕后，客户端从距离最近的存放第二个数据块的DataNode中读取数据，以此类推，读完全部。
- 读取时，客户端可一次性获取所有文件块的位置信息
- 写入时，客户端在每次写入文件块之前都要询问NameNode该文件块存放的位置：

HDFS中节点的存活状态、负载均衡等情况在文件块写入后都会发生变化，因为NameNode无法预先决定所有文件块的存放位置。

对于读，客户端只需找最近的DN读即可，无需从所有的DN中读。

- 文件读写与一致性

- 单机策略：加锁——编程复杂
- “一次写入，多次读取”
 - 一个文件经过创建、写入和关闭后即不得改变文件中的已有内容
 - 若已经写入HDFS文件，则仅允许在用户末尾追加数据，即执行append操作
 - 当对一个文件执行写入或者追加操作时，NameNode拒绝其他针对该文件的读写请求
 - 当对一个文件执行读取操作时，NameNode允许其他针对该文件的读写请求
- 即，想修改HDFS文件中的内容，需要先删除该文件才能重新写入文件。但是用户编程时无需考虑文件的并发读写问题。因为若允许修改HDFS文件内容，则表示可以更改文件块中的内容，那么文件块的大小等元数据可能也会随之变化，且文件块的多个副本之间统一需要保持数据的一致性，而分布式系统之间副本的同步是很复杂的。

容错机制

- NameNode故障

- Secondary NameNode备份
- 只是冷备份，可能会造成部分文件修改操作的丢失。
- DataNode故障
 - DN会定期向NN发送心跳表示该节点处于活跃状态，若没法，NN会认为该DN处于宕机状态，该DN的所有数据均被视为不可读，所有客户端申请的读写请求NN都会排除该DN。
 - HDFS中的文件均有若干副本，一旦有DN发生宕机，则将导致该部分的文件块的副本数量小于预设，此时系统需要在其他DN上对该类文件块进行备份，以达到预设数量。

编程示例

- shell命令

- JAVA程序

○ 步骤

- 获取HDFS文件系统对象，类型为FileSystem
- 利用FS对象创建HDFS文件的输入流或输出流对象，类型为FSDataInputStream或FSDataOutputStream
- 利用输入流或输出流对象，实现针对HDFS文件的操作

○ 写文件、读文件

```

7
8 public void write(String hdfsFilePath, String localFilePath) throws IOException {
9     /* 步骤 1: 获取 HDFS 的文件系统对象 */
10    Configuration conf = new Configuration();
11    FileSystem fs = FileSystem.get(URI.create(hdfsFilePath), conf);
12    /* 步骤 2: 获取输出流 hdfsOutputStream */
13    FSDataOutputStream hdfsOutputStream = fs.create(new Path(hdfsFilePath));
14    /* 步骤 3: 利用输出流写入 HDFS 文件 */
15    // 读取本地文件的输入流
16    FileInputStream localInputStream = new FileInputStream(localFilePath);
17    // 将本地文件的输入流复制到 HDFS 文件的输出流
18    IOUtils.copyBytes(localInputStream, hdfsOutputStream, 4096, true);

```

```

7
8 public void read(String hdfsFilePath, String localFilePath) throws IOException {
9     /* 步骤 1: 获取 HDFS 的文件系统对象 */
10    Configuration conf = new Configuration();
11    FileSystem fs = FileSystem.get(URI.create(hdfsFilePath), conf);
12    /* 步骤 2: 获取输入流 hdfsInputStream */
13    FSDataInputStream hdfsInputStream = fs.open(new Path(hdfsFilePath));
14    /* 步骤 3: 利用输入流读取 HDFS 文件 */
15    // 写入本地文件的输出流
16    FileOutputStream localOutputStream = new FileOutputStream(localFilePath);
17    // 将 HDFS 文件的输入流复制到本地文件的输出流
18    IOUtils.copyBytes(hdfsInputStream, localOutputStream, 4096, true);
19 }
20 ...

```

表 2.2 HDFS Shell 命令

命令	用法示例	含义
ls	hadoop fs -ls <文件夹路径>	列出文件夹下的所有内容
cat	hadoop fs -cat <文件路径>	查看文件内容
mv	hadoop fs -mv <源路径><目的路径>	将源路径移动到目的路径
mkdir	hadoop fs -mkdir <文件夹路径>	创建文件夹
rm	hadoop fs -rm <文件路径>	删除文件
rmdir	hadoop fs -rmdir <文件夹路径>	删除空文件夹
put	hadoop fs -put <本地文件路径><HDFS 文件路径>	将本地文件写入 HDFS
get	hadoop fs -get <HDFS 文件路径><本地文件路径>	将 HDFS 文件读取到本地

习题

- 1.举例说明HDFS在实际应用中的作用
 - 存储TB级别的大文件
 - 保证文件系统的容错

- 实现大文件的并发读写控制
- 2.HDFS的主要部件有哪些？各个部件分别有什么作用？
 - NameNode：主节点负责管理目录结构、位置等元数据，维护DN状态等，不实际存储文件
 - Secondary NameNode：主节点的备份，故障时恢复
 - DataNode：存储文件，根据NN的控制信息存储和管理对于的文件块，定期向NN汇报自己的状态
- 3.HDFS的文件分块是什么？和OS的文件系统中块的概念有何联系和区别？
 - HDFS将文件分成若干个固定大小的块，块的大小用户可以指定，然后分布存储于DN中，HDFS的分块单纯是数据分块，优化分布式读取，减少寻址开销。OS中，块的大小通常是固定的，远小于HDFS的分块，文件分块是系统层面指定的，目标优化磁盘IO性能，减少磁盘碎片，让系统更好地工作。
 - 两者的文件分块都是为了更好地管理数据。
- 4.简述客户端从HDFS中读取某一文件时的工作过程
 - 客户端和NN通信发送请求
 - NN判断路径等合法信息，合法则返回该文件所有数据块的存放地址
 - 客户端依次读取数据块，均从存放每数据块的最近的DN中读取，直至读完。
- 5.HDFS为什么要“一次写入，多次读取”？
 - 因为若允许修改HDFS文件内容，则表示可以更改文件块中的内容，那么文件块的大小等元数据可能也会随之变化，且文件块的多个副本之间统一需要保持数据的一致性，而分布式系统之间副本的同步是很复杂的。
 - 一次写入，写时防止其他的读写，保证数据读写的一致性
 - 多次读取，对一个文件执行读取操作时，NameNode允许其他针对该文件的读写请求
- 6.为什么HDFS按照逐个进行的阻塞方式读写文件块？能否并行读写文件块？
 - 客户端向HDFS读写文件时，NN需先定位到需要读写的位置反馈给客户端，客户端给DN传输文件数据包时，DN之间也在传输该数据文件，待完成该数据块的所有数据包的分布式备份后，再申请下一数据块的读写。
 - 可以并发读，不能并发写。只有一个客户端可以追加写，写时防止其他的读写，阻塞可保证数据块在不同的DN上有一致的副本冗余存储，保证数据读写的一致性。

3. MapReduce

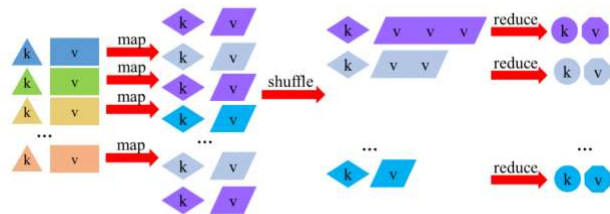
设计思想

- MPI与MapReduce

- MPI提供了分布式环境下的一种并行编程范式：用标准串行语言书写的代码加之用于消息接受和发送的库函数调用。
- MPI缺点：
 - 用户编程角度：程序员需要考虑进程之间的并行问题，切进程间的通讯需要用户在程序中显式表达，增加编程复杂性。
 - 系统实现角度：MPI程序以多进程方式运行，除非用户编写程序时添加了故障恢复功能，否则MPI本身不提供容错功能——故障崩溃，计算资源的浪费。

- 数据模型

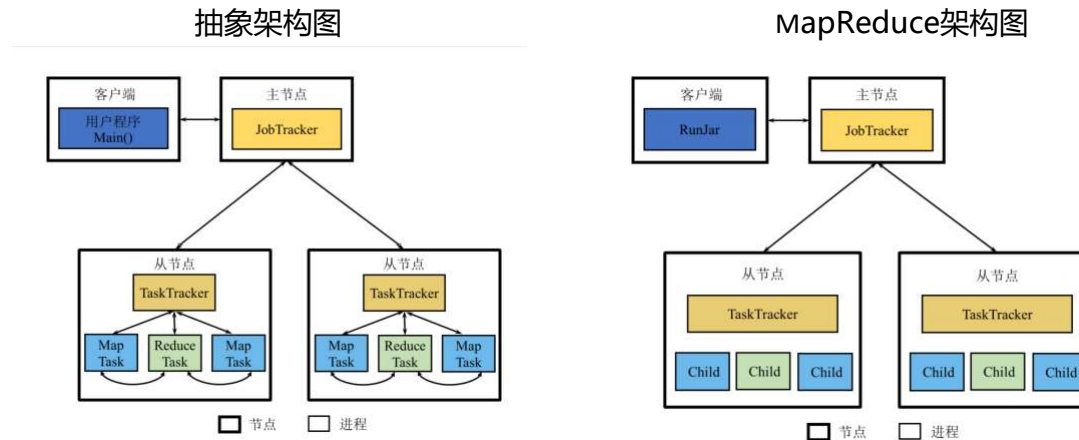
将数据抽象为一系列键值对，在处理过程中对键值对进行转换



- 计算模型

- MR将并行计算过程抽象为Map和Reduce两个过程，采用分而治之的策略对数据进行并行处理
- 逻辑计算模型
 - 抽象为M和R两个过程：M对输入键值对进行一次变换，产生若干个新的键值对；R对相同键的键值对进行计算，并可根据需要将计算结果进行一次键值对转换后输出

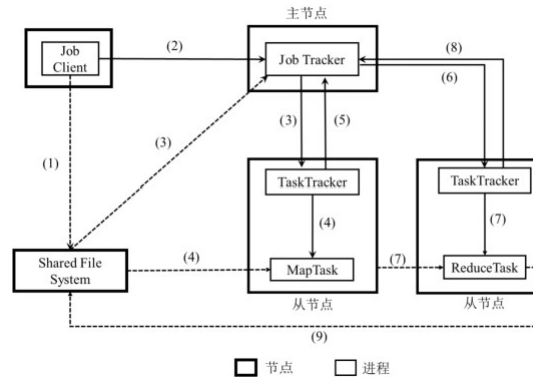
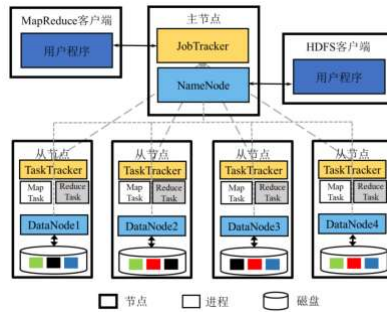
- DAG有向无环图
- 物理计算模型
 - 分而治之，由多个任务并行处理
- 用户编程容易：不需要掌握分布式并行编程细节
- 体系架构
 - 架构图



- JobTracker: 主节点运行的后台进程
 - 负责整个系统的资源管理和作业管理
 - 资源管理：监控TaskTracker来管理系统拥有的计算资源
 - 作业管理：负责将Job拆分为Task，并进行任务调度/跟踪任务的运行进度、资源使用量等
- TaskTracker: 从节点运行的后台进程
 - 负责管理本节点的资源、执行JobTracker命令、汇报情况。（slot等量划分资源/心跳汇报/执行任务等）
- Task: 从节点在应用程序执行过程中启动的进程
 - 负责任务执行。JT根据TT汇报的信息进行调度，命令存在空闲的TT启动Task进程执行M或R任务。
 - HadoopMapReduce架构中，**该进程名为Child**
- 客户端：客户端所在节点为提交应用程序启动的进程
 - 负责将用户编写的MR程序交给JT。HMR中，**该进程名为RunJar**
- 计算向数据靠拢的思想



- 计算向数据靠拢的思想



- 应用程序执行流程（上右图）

- Client将用户编写的MR配置信息等上传到共享文件系统HDFS
- Client提交作业给JT，告知作业信息位置
- JT读取作业信息，生成一系列M和R任务，调度给拥有空闲slot的TT。
- TT根据JT的指令启动child进程执行M任务，从HDFS中读取输入数据
- JT从TT获得关于M任务进度的信息
- M任务完成后，JT分配R任务给TT。
- TT根据JT的指令启动child进程执行R任务，从M任务所在节点的本地磁盘拉取M的输出结果。
- JT从TT获得关于R任务进度的信息。
- R任务完成并将结果写入HDFS，整个作业执行完毕。

• 工作原理

- 数据输入

- 根据输入文件的格式定义到键值对的映射
- 多个并行Map任务一起：如何划分——文件分块存储，根据块的数量启动Map任务，每个块对应每个Map？
- 可能存在跨块记录：定义**输入格式InputFormat**，用于描述输入数据的格式，提供数据逻辑划分和键值对解析功能。
 - IF将数据在逻辑上划分为若干切片split，包含一些元数据，如数据起始位置/数据长度/数据所在节点的信息等。

- Map任务读取的单位是分片，分片的数量往往也决定了Map任务的个数。
- IF还提供键值对解析的方法，即给定一个分片，输入格式将根据分隔符/大小等元数据将分片中的数据解析为相应键值对
- IF通过定义分片和键值对解析的方法完成从物理存储（HDFS文件块）到MapReduce程序可处理的逻辑数据之间的映射。

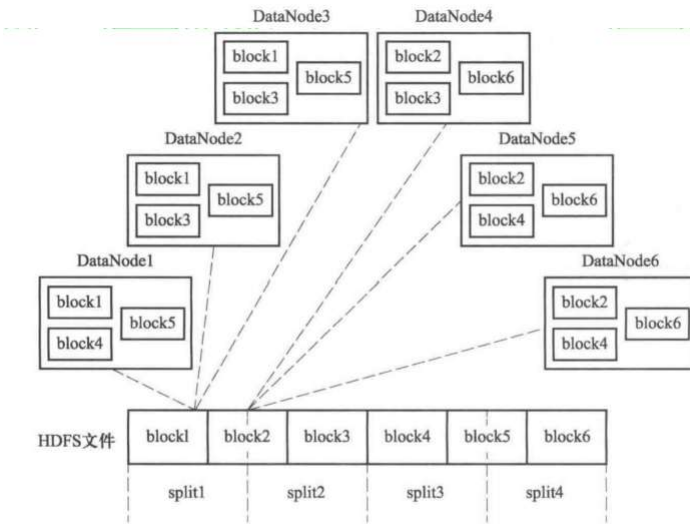


图 3.10 InputFormat 中的 split 与 HDFS 中的 block

表 3.1 常见的输入格式

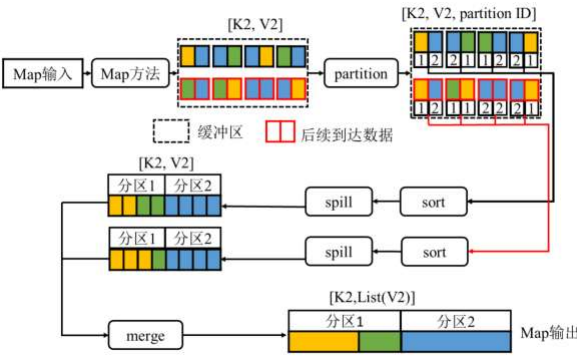
名称	含义
TextInputFormat	MapReduce 中默认的输入格式，该格式将输入文件中的行偏移量和行的内容作为输入键值对，键值对对应的数据类型为 LongWritable 和 Text
KeyValueTextInputFormat	该格式将输入文件中的每行内容按分隔符切分为键值对，默认的分隔符为制表符，对应的数据类型为 Text 和 Text
NLineInputFormat	该格式与 TextInputFormat 一致，将输入文件中的行偏移量和行的内容作为输入键值对。同时，该格式指定了每个 Map 任务接收的输入行数 N，默认的 N 值为 1，即每个 MapTask 正好接收 1 行输入

名称	含义
CombineTextInputFormat	该格式将多个文件划分到一个切片中，从而将多个文件交由一个 Map 任务处理。采用该格式的输入键值对与 TextInputFormat 和 NLineInputFormat 一致
SequenceFileInputFormat	MapReduce 内置的 SequenceFile 类提供了存储二进制键值对的数据结构。SequenceFileInputFormat 用于处理以该种数据结构存储的顺序文件，从而解析出键值对。键值对的数据类型由顺序文件中保存的二进制键值对的数据类型决定
DBInputFormat	该格式用于从关系型数据库中读取数据，并将元组序号和元组作为输入键值对。其中元组序号的数据类型为 LongWritable，而元组的数据类型需要由用户继承 DBWritable 后自定义

- Map

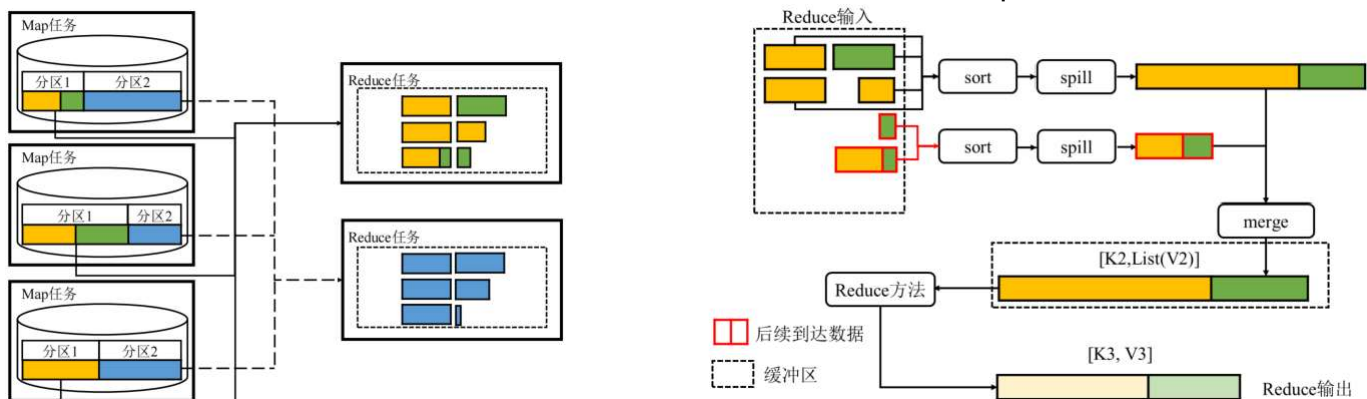
- 对一个键值对，M将其转换为一个或多个键值对
- M将输入映射为[K2,V2]后根据partition方式确定其所属分区
- 总分区数 = R任务数
- 得到 ([K2,V2],partitionID) 后将其写入缓冲区，
- 缓冲区达阈值，M锁定当前阈值内的缓冲区，按照partitionID排序，同一分区内按键排序，聚集发往同一R任务的键值对。
- combine方法：每个map任务内将键相同的键值对合并计数。
- 同一Map任务通常会**产生多个磁盘文件**，数量达到一定阈值后，Map会进一步归并为一个文件，使得属于同一分区的所有键值对连续存储，最终输出以一个大文件的形式存储于Map任务所在节点的本地磁盘中。

- Shuffle



○ 何时Shuffle

- 当系统中的Map任务完成率达到阈值，系统将启动Reduce任务
- Reduce的slow start。但R拉取的数据必然来自于已完成运行的Map任务，即已保存在磁盘上的文件



- Reduce

- 对一个键值对，R将其转换为一个或多个键值对
- R拉取得到的数据首先放入内存缓冲区，达到预定阈值后，R对缓冲区内键值按键进行归并排序
- M用了combine则优先合并。
- 将内存缓冲区的数据溢写到磁盘文件，数量达到一定阈值后，Reduce会进一步归并为一个文件，文件中的键值对按键排序。
- R最后一次归并无需再次写入磁盘，而是直接通过reduce方法处理，从而省略一次磁盘读写（不用写到本地）。最终R的结果写入用户程序指定的存储系统，通常为HDFS。
- 任务数量：程序指定/最优个数取决于集群中可用的reduce任务槽的数量/通常设置比槽小一点的个数

- 数据输出

- 定义输出格式OutputFormat，完成从MR程序处理的逻辑键值对数据到物理存储之间的映射。

表 3.2 常见的输出格式

名称	含义
TextOutputFormat	MapReduce 中默认的输出格式，该格式将接收到的键值对写入文本中的每一行，写入的键值对可以是任意类型
NullOutputFormat	该格式忽略接收到的键值对，不进行任何输出
LazyOutputFormat	使用 TextOutputFormat 可能产生一些空的输出文件，而使用 LazyOutputFormat 可避免空输出文件的产生
SequenceFileOutputFormat	该格式将输出键值对以 SequenceFile 类提供的数据结构输出到文件
DBOutputFormat	该格式将输出键值对存储到关系型数据库

- 容错机制

- JT故障

- 对于MR1.0, JT故障意味着所有作业需要重新执行
 - 没有处理故障机制，成为单点瓶颈

- TT故障

- JT收不到心跳，认为该TT故障，该TT所在节点运行过的任务均被标记为失败，JT重新把任务调度到别的TT执行。此过程对用户透明，仅表现为作业执行速度降低

- Task故障

- 代码错误/磁盘数据损坏/节点暂时性故障等。
 - TT检测到后在下一次心跳时汇报给JT，JT收到消息重新调度执行该任务，该任务可能在集群中的任意一个节点重试。
 - 若某个任务经过最大尝试数的尝试运行后扔失败，整个作业标记为失败。
 - M：从HDFS重新读数据
 - R：从M处磁盘里重新拉，如果无法读取/磁盘数据丢失，M也得重做

- 编程示例

- Map/reduce主要步骤

- 确定输入和输出键值对的数据类型
 - 在map方法中编写处理逻辑，将K1V1转换为K2V2输出/在reduce方法中编写处理逻辑

- 主方法步骤

- 首先在run方法中设置作业的信息如程序类名/数据的输入输出路径格式/mr方法/输出键值对的数据类型等
 - main方法中使用MR提供的工具类ToolRunnew运行作业。

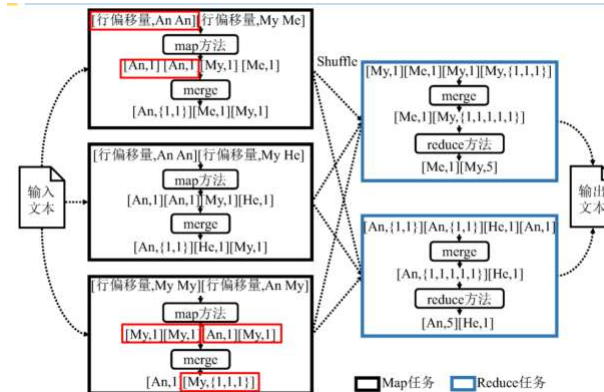
未设置数据的输入输出格式，则作业默认采用TIF和TOF作为输入输出格式。

- 词频统计

- map实现分词，reduce对相同单词进行计数
- combine能够降低M和R任务消耗的存储空间，且有利于降低shuffle过程中传输的数据量

- 关系表自然连接及其优化

- map需要对元组进行标识，并输出以共同属性列为键的键值对，reduce需要执行连接操作。
- 自定义数据类型ReduceJoinWritable：M的输出R的输入，包含data记录元组数据，tag标识哪种表
- 问题：表格一大一小——分布式缓存广播小表，自然连接在M即可完成，避免了shuffle，提升整体性能



- 网页链接排名

- 迭代计算：每次迭代中，网页排名通过计算每个网页的排名值，并更新相应网页信息的排名值，更新后的排名值将在下一次迭代中参与计算和更新。直至满足终止条件（更新前后插值小于阈值或迭代次数达标）
- 无法通过一个MR作业完成整个迭代计算——一个MR实现一次迭代。
- map产出两组键值对并标识，reduce计算并更新网页的排名值，通过重复运行同一个MR作业实现迭代的计算。

- K均值聚类

• 总结

- 设计思想：数据模型——键值对
- 工作原理：数据分片解析为键值对，Map转化，Reduce输出
- 容错：JT只能重启，TT重新调度任务，Task重新调度
- 一定程度缓解了大规模数据处理的困难
- 缺陷：
 - eg关系表连接需要显式定义tag标记信息来自哪种表，增加了用户编程的复杂性
 - 迭代只能重复开启作业，静态数据会重复加载，并且多个MR作业之间进行数据传输需要反复读写HDFS等文件系统，导致不必要的IO开销。

习题

- 1.MR相比MPI的优势？

用户：简化编程，MPI需要用户手动进行程序并行的设计

容错：MPI自身没容错保证。MR有一定的容错（JT无，TT和Task有）

- 2.解释MR的逻辑计算模型和物理计算模型

- 逻辑计算模型：Map和Reduce两个算子，Map——>Reduce，仅有两个顶点的DAG。
- 物理计算模型：M和R两个逻辑算子在物理上需要若干个实例来实现，一个算子可以同时由多个实例并行执行。

- 3.MR的主要部件有哪些，分别有什么作用？

- JT：主节点运行的管理进程，负责系统的资源管理和作业管理，将job拆成task
- TT：从节点运行的后台进程，管理单个节点上的资源和任务，使用slot等量划分节点上的资源，向JT汇报
- Task (Child)：执行M或R任务。
- Client (RunJar)：向主节点提交启动应用程序的进程

- 4.如果不考虑数据输入和输出阶段，MR的工作阶段可以划分为哪些阶段，各个阶段主要完成哪些功能？

- Map：将一个键值对转化为一个或多个键值对，依据partition以键对数据进行划分，决定交由哪个Reduce处理。当缓冲区达到阈值后，锁定当前阈值的数据，按照键排序发给Reduce。
另外可以定义combine方法，在每个Map节点上将键相同的键值对合并，压缩数据减少IO。
Map将排序后的结果溢写到磁盘形成文件，溢写的文件数达到阈值后Map会进一步归并为一个文件。
- shuffle：把键相同的键值对发给同一个R任务。
- Reduce：将一个键值对转化为一个或多个键值对，从M节点磁盘处拉取数据，放入内存缓冲区，达到一定阈值后按键进行归并排序，归并后的数据在缓冲区达到一定阈值后溢写到本地磁盘。溢写文件达到阈值后执行归并合成大文件。大文件直接写入HDFS中，无需再保存到R的节点磁盘上，省略一次磁盘读写。

- 5.简述MR与HDFS的关系

- MR和HDFS都是Hadoop项目下的子项目，一个是分布式计算系统，一个是分布式存储系统。MR的输入和输出都可以来自HDFS和输出到HDFS上。在实际任务中，M会从读写开销最小的DN中读取数据，R也会写到更近的DN中减少网络开销，MR体现了计算向数据靠拢的思想。

- 6.MR中的归并merge和合并combine的区别？

- 归并是将多个文件合成一个，比如Map和Reduce输出时溢写的文件会最终归并为一个文件
- 合并是为了减少磁盘IO的开销，map过程中使用combine可以在每个map节点上先合并键相同的键值对，压缩数据，有效减少磁盘的IO。
- 7.MR中的分布式缓存机制有什么作用？
 - 某些情况下，分布式缓存机制可以提高程序的运行效率，比如进行关系表的自然连接任务中，若表一大一小，可以通过缓存机制广播小表，在map阶段就直接完成连接，提高程序的运行效率。
 - 同时，分布式缓存机制也可以在单个节点突然宕机的时候从别的节点找到该节点上同样的副本，保证数据的一致性和高可用性。
- 8.若MR以HDFS中的文件作为输入，那么IF中的split和HDFS中的block是否必然一一对应，为什么？
 - 否。物理分块block的依据是数据块，而split是逻辑上的分片，如文件的一句话被划分在不同的物理块上，但split的读取逻辑是整行读取，此时split就可能对应多个block。
- 9.若MR运行过程中仅有一个R的进程崩溃而其他部件正常，那么MR会重启一个R任务，重启后的R任务从何处获取数据？
 - 读取该任务对应的Map任务所在节点的本地磁盘中保存的Map任务的输出文件。若该文件无法读取，则需要重启Map任务，等待Map把最终的输出文件保存到磁盘中再读取。
- 10.假设Map任务的缓冲区无限大，是否仍有必要溢写磁盘，类似地，若Reduce任务的缓冲区无限大，是否仍需溢写磁盘？为什么？
 - M需要，溢写也为了容错，R需要从M的输出本地磁盘中读取数据，若不写到磁盘，R将无法获取数据
 - R可以不溢写，缓冲区内完成任务后可直接写到HDFS的指定输出位置。若故障需重启，可从M处重新获取数据，并且重新分配时也不能保证是否还在原来的节点，溢写没有意义。

4. Spark

简介

- 由美国加州伯克利大学的AMP实验室开发
- 最初是基于内存计算的批处理系统，用于构建大型的低延迟的数据分析应用程序
- 逐步发展为内外存同时使用的批处理系统，并增加了sparkStreaming支持实时流计算，Structured Streaming 支持批流融合

设计思想

- MR的局限性
 - 编程框架的表达能力有限，用户编程复杂
仅提供M和R两个算子，用户需要基于二者实现数据处理的操作，增加了编程难度。
 - 单个作业中需要进行shuffle的数据以阻塞方式传输，磁盘IO开销大、延迟高
单个作业中，需要进行shuffle的数据先由M任务将计算结果写入磁盘，之后R才可读取该计算结果，因此需要进行Shuffle的数据磁盘IO开销大，阻塞式数据传输也加剧了高延迟。
 - 多个作业之间的衔接涉及IO开销，应用程序的延迟高
很多程序需要通过多个作业完成，迭代计算等，前一个作业需要将迭代计算结果写入HDFS，后一个作业从HDFS中读取该结果再进行下一轮迭代，反复读写加剧了高延迟。
 - 优化方向：提高编程框架的表达能力、减少不必要的shuffle、消除多个衔接作业间的IO
- 数据模型
 - spark将数据抽象为弹性分布式数据集RDD
 - Resilient：弹性，具备可恢复的容错特性
 - Dataset：数据集，MR操作的是文件，Spark为数据集
 - Distributed：分布式的，每个RDD可划分成多个分区，每个分区即一个数据集片段，一个RDD的分区可以存储于集群中的不同节点
- 计算模型
 - spark提供丰富的算子体系架构
 - 创建：从本地内存或外部数据源创建RDD，数据输入

- 转换：描述RDD的转换逻辑，提供对RDD进行变换的功能
- 行动：标志转换结束，触发DAG生成

表 4.1 Spark 中常用的 RDD 创建操作算子

创建操作	含义
parallelize(seq, [numSlices])	从内存集合创建 RDD
textFile(path, [minPartitions])	读取 HDFS 兼容的文件系统下的文件以创建 RDD
wholeTextFiles(path, [minPartitions])	读取 HDFS 兼容的文件系统下的文件夹中的所有文件以创建 RDD
hadoopFile(path, inputFormatClass, keyClass, valueClass, [minPartitions])	读取 HDFS 兼容的文件系统下拥有任意 inputFormat 的文件以创建 RDD

表 4.2 Spark 中常用的 RDD 转换操作算子

转换操作	含义
map(func)	对 RDD 中的每个记录均使用 func 进行转换，返回一个新的 RDD
filter(func)	过滤对 RDD 中的每个记录使用 func 后返回值为 true 的记录
flatMap(func)	与 map 类似，但可将 RDD 中的每个记录映射为 0 个或多个新记录
mapPartitions(func)	与 map 类似，但对每个分区进行操作
union(otherRDD)	两个 RDD 取并集得到一个新的 RDD
intersect(otherRDD)	两个 RDD 取交集得到一个新的 RDD
groupByKey([numPartitions])	将[K,V]键值对按键分组，返回一个由[K, Iterable<V>]组成的新的 RDD
reduceByKey(func, [numPartitions])	将键值对按键聚合，在每个键的所有值上使用 func，返回一个由[K, V]组成的新的 RDD
sortByKey([ascending], [numPartitions])	将键值对按键排序，返回一个新的 RDD
join(otherRDD, [numPartitions])	[K, V1]和[K, V2]分别属于两个 RDD，返回一个由[K, (V1, V2)]组成的 RDD
cogroup(otherRDD, [numPartitions])	[K, V1]和[K, V2]分别属于两个 RDD，返回一个由[K, (Iterable<V1>, Iterable<V2>)]组成的 RDD

表 4.3 Spark 中常用的 RDD 行动操作算子

行动操作	含义
reduce(func)	将 RDD 中的记录按 func 聚合，该 func 必须满足交换律和结合律
collect()	将 RDD 中的所有记录收集到 driver 中，返回一个 Array
count()	返回 RDD 中记录的个数
first()	返回 RDD 中的第 1 个记录
take(n)	返回 RDD 中的前 n 个记录
saveAsTextFile(path)	将 RDD 中的记录以文本文件的形式写入本地文件系统、HDFS 或其他任何 Hadoop 支持的文件系统中的给定目录
countByKey()	按 key 统计计数，返回一个由[K, Long]组成的 map
foreach(func)	对 RDD 中的每个记录均使用 func

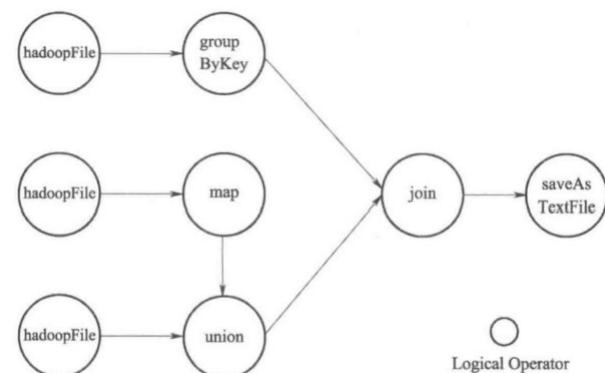


图 4.2 Spark 逻辑计算模型：Operator DAG

○ 逻辑计算模型

▪ 分类

- 算子操作角度，如上右图
- RDD变换角度，这俩的区别上边的主体为算子，变换角度主题是数据

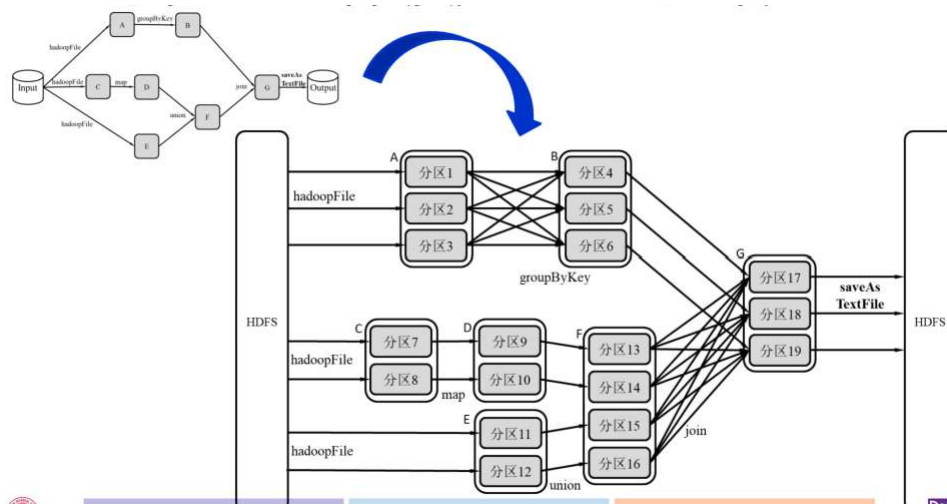
▪ RDD Lineage

- RDD读入外部数据源进行创建
- RDD经过一系列的转换操作，每次都会产生不同的RDD，供给下一个转换操作使用
- 最后一个RDD经过动作操作进行转换，并输出到外部数据源

- spark系统保留RDD Lineage的信息
 - 容错，某节点上的RDD丢失，只需顺着依赖链计算丢失部分上游的RDD
 - 减少磁盘IO
- RDD性质
 - RDD是只读的记录分区的集合，经创建后，不能进行修改
 - 不可变，通过在其他RDD上执行确定的转换操作而得到新的RDD，而不是改变原有的RDD
 - 遵循函数式编程的特性，变量的值是不可变的

○ 物理计算模型

- 每个Task通常负责处理RDD的一个分区



• 体系架构

- 架构图

○ 抽象架构图

- Cluster Manager集群管理器
 - 管理整个系统的资源并监控工作节点
 - 部署方式分类
 - ◆ standalone: CM中包含Master和Worker
 - ◆ Yarn: 与资源管理系统Yarn共同部署，CM中包含ResourceManager和NodeManager

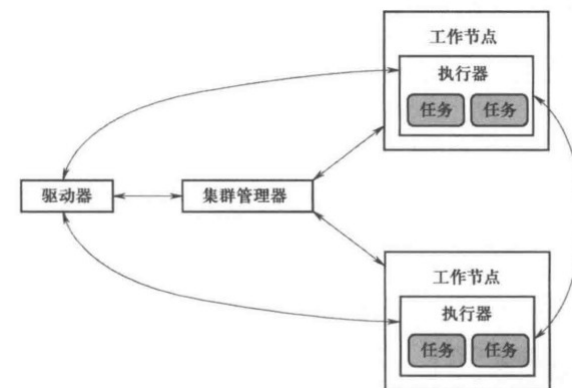


图 4.6 Spark 抽象架构图

- **executor执行器**
 - 负责任务执行，执行器本身为运行在工作节点上的一个进程，通过启动若干线程Task或线程组TaskSet执行任务。
 - st模式下执行器进程名为CoarseGrainedExecutorBackend
- **driver驱动器**
 - 负责启动应用程序的主方法并**管理作业运行**
 - 逻辑上独立于主节点从节点和客户端，但会根据应用程序的不同运行方式以不同的形式存在

spark的架构实现了资源管理和作业管理两大功能的分离

○ standalone下分类

- **client模式**
 - 驱动器位于客户端，处于sparkSubmit进程中
 - 客户端可以查看应用程序执行过程中的信息
- **cluster模式**
 - Master选择在某一Worker所在节点启动名为DriverWrapper的进程作为驱动器
 - 客户端无法查看应用程序执行过程中的信息

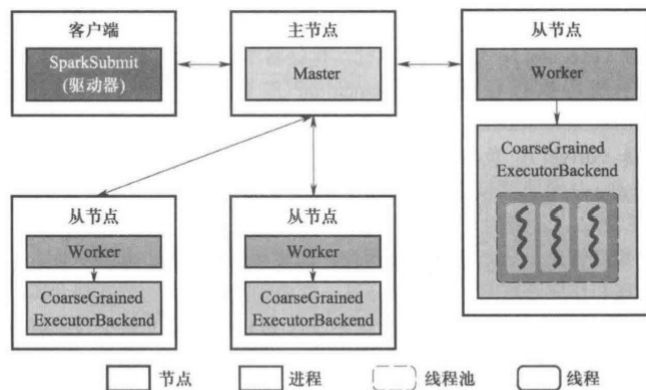


图 4.8 Standalone Client 模式下的 Spark 架构图

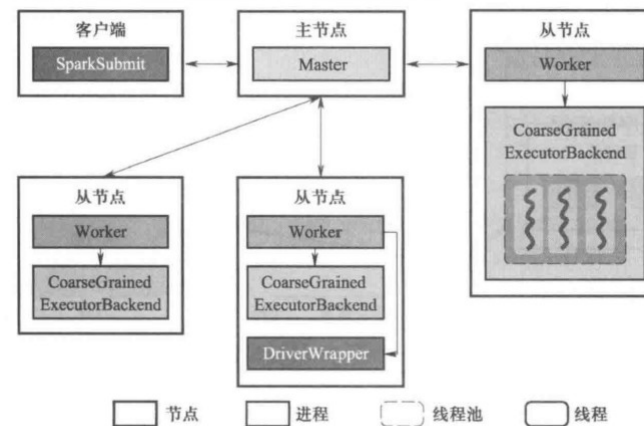


图 4.9 Standalone Cluster 模式下的 Spark 架构图

○ 与MR的比较

- **任务执行**
 - MR采用多进程，child进程执行任务代码Task；
 - spark采用多线程，Task任务代码同时以工作线程形式存在

- 基础接口
 - MR只有M和R
 - spark提供很多基于RDD编程接口，趋向于声明式编程风格。

表 4.4 MapReduce 和 Spark 对比

系统	MapReduce	Spark
系统进程	JobTracker	Master
	TaskTracker	Worker
	Child	CoarseGrainedExecutorBackend
工作线程	—	Task
任务代码	Task	
基础接口	map/reduce	RDD API

- 应用程序执行流程
 - 启动Driver: st模式下, Client模式, 客户端直接启动驱动器, 向master注册; Cluster模式, 客户端向Master提交应用程序, Master选择一个Worker启动DriverWrapper。
 - 构建基本运行环境: 由Driver创建SparkContext, 向Master申请资源, 由Driver进行任务分配和监控
 - Master通知Worker启动执行器进程CGEB, 该进程内部以多线程方式运行任务
 - CGEB向Driver驱动器注册
 - sparkContext构建DAG并进行任务划分, 从而将其交给CGEB执行器进程中的线程以执行任务。
- 工作原理

spark中, Driver举足轻重, 其创建的SparkContext维护了应用程序的基本运行信息

 - SC首先根据RDD的依赖关系构建DAG, 由DAG调度器划分DAG stage, 将一个Stage作为一个TaskSet提交给任务调度器, 有其将任务分发给各个Worker上的CGEB执行器执行, 整个过程按照Stage依次执行

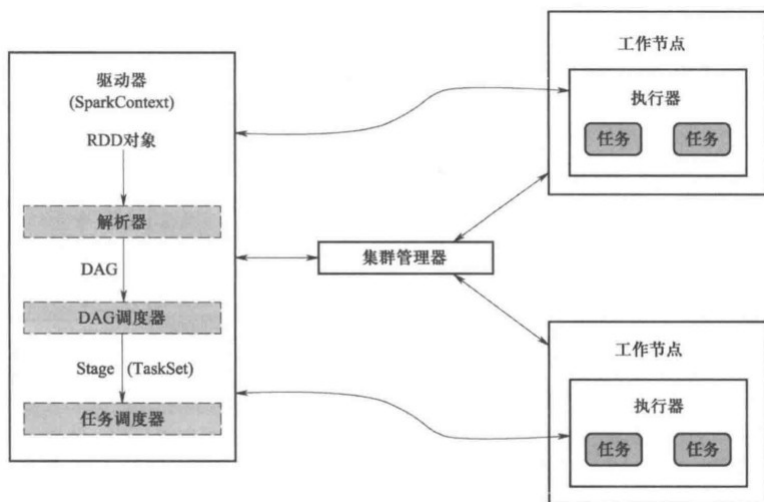


图 4.11 驱动器 (SparkContext) 内部工作过程

- stage划分

○ 依赖关系

- 窄依赖：父RDD的每个分区最多被一个子RDD的分区所用（1对1/多对1）
- 宽依赖：子RDD的分区依赖于父RDD的多个分区或所有分区（1对多）

○ 划分

- DAG调度器通过分析各个RDD中分区之间的依赖关系决定如何划分Stage
 - 宽依赖生成新的Stage
 - 窄依赖则将该Operator加入当前Stage
- Stage内部为窄依赖，Stage之间为宽依赖
- 只有stage之间的数据传输需要进行Shuffle。
- 根据输出不同分类
 - shuffleMapStage：以Shuffle为输出，作为另一个Stage的输入；输出必须经过Shuffle；一个DAG中可有可无
 - ResultStage：直接产生最终的结果；一个DAG中必然包含此类Stage

- 一个DAG含有一个或多个Stage，其中至少含有一个ResultStage

- stage内部数据传输

- 所有依赖都是窄依赖，可以直接Pipeline流水线传输

- 与MR中shuffle区别
 - 不需要物化（序列化保存到磁盘中），流水线方式不要求物化前序算子的所有计算结果，并且无需等待其他分区操作结束。
 - spark根据每个Stage输出RDD中的分区个数决定启动的任务数量，以执行该Stage中的操作。
- 分类
 - 若stage类型为ShuffleMapStage，则任务称为ShuffleMapTask；
 - 若stage类型为ResultStage，则任务称为ResultStage。
- stage之间数据传输
 - 可能发生在两个SMS之间，或SMS和RS之间
 - Task角度，可能发生在两个SMT之间或SMT和RT之间。
 - 两阶段
 - shuffle Write：SMT将输出RDD记录按Partition函数划分到相应的bucket中并物化到本地磁盘形成ShuffleblockFile
 - Shuffle Read：SMT或RT根据partition函数读取相应的ShuffleblockFile，存入buffer并进行后续计算

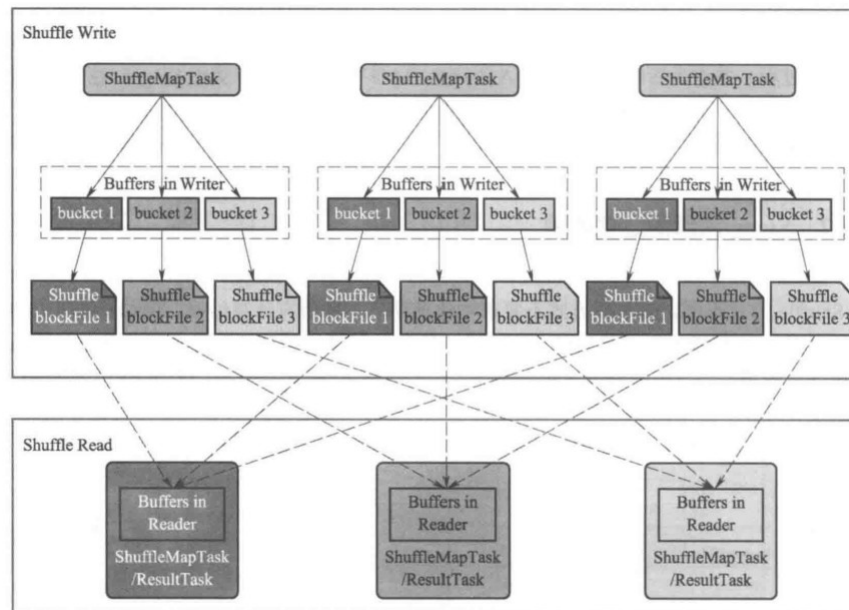


图 4.16 Stage 之间的数据传输

- 应用与作业
 - Application：用户编写的Spark应用程序

- Job (DAG)：一个Job包含多个RDD以及作用于相应RDD转换的操作，其中最后一个为action
- MR中一个应用就是一个作业；Spark中一个应用可以由多个作业完成
- Stage/TaskSet：一个Job会分为多个Stage/TaskSet
 - 若干窄依赖的RDD操作
 - Job的基本调度单位
 - 代表了一组关联的、相互之间没有shuffle依赖关系的任务组成的任务集
- Task：运行在Executor上的工作单元

表 4.5 从逻辑执行和物理执行两个角度理解 Spark 应用

类别	逻辑执行角度	物理执行角度
Application	DAG	Job
	Stage	TaskSet
	RDD Operator	Task

• 容错机制

- 运行故障

- Client故障
 - standalone Client：Driver在客户端，用户需重新提交应用程序
 - Standalone Cluster：作业成功提交，则不会影响系统；若DriverWrapper故障，需用户重新提交
- Master故障：重启/ZooKeeper实现高可用性
- Worker故障：重启/重新分配
- Executor故障：重启/重新分配

- RDD持久化

- 问题：
 - 计算过程中会不断产生新的RDD，系统不能将所有RDD都存在内存
 - 一旦达到阈值，spark会使用置换算法将部分RDD内存空间腾出
 - 若不做任何声明，这些RDD会被直接丢弃，但某些RDD在后续有可能被再次使用
- 存储机制：RDD提供持久化的接口
 - persist (StorageLevel)
 - 接受StorageLevel类型参数，可配置各种级别

- 持久化后的RDD会保留在工作节点中，可重复使用
 - StorageLevel
 - ◆ 1.MEMORY_ONLY: JVM中缓存，内存不足直接丢弃
 - ◆ 2.MEMORY_AND_DISK: JVM中缓存，内存不足写入磁盘
 - ◆ 3.MEMORY_ONLY_SER: 为内存中每个分区存储一个byte数组，数组内容为当前分区中Java对象的序列化结果
 - ◆ 4.MEMORY_AND_DISK_SER: 与MEMORY_AND_DISK类似，每个分区存储序列化后组成的byte数组。
 - ◆ 5.DISK_ONLY: 将每个分区的数据序列化到磁盘中
 - ◆ 6.MEMORY_ONLY_2: 与MEMORY_ONLY一样，每个分区备份到两台机器上
 - ◆ 7.MEMORY_AND_DISK_2: 与4相同，每个分区备份到两台机器上
 - cache: 缓存
 - 相当于persist (MEMORY_ONLY)
 - 用户可以通过缓存RDD加快计算速度，并利用多个备份快速恢复因故障丢失的数据分区，因此，用户进行显式持久化的功能是多方面的。
 - 系统在RDD计算过程中也会进行自动（隐式）持久化，即自动保存部分shuffle过程的数据。
- 故障恢复
 - Driver的SparkContext维护了记录RDD转换操作的DAG，即RDD的Lineage。
 - 相比数据库的log，这里记录的是粗粒度的变换操作
 - 当某一RDD部分分区丢失时，可通过Lineage获取足够的信息以重新运算和恢复丢失的数据分区
 - 结合计算过程中某些持久化的RDD和Lineage信息即可恢复。
 - 重算过程在不同节点之间可以并行
 - 窄依赖等待父RDD即可；宽依赖需等待父所有RDD都恢复
- 检查点
 - 问题：Lineage可能非常长；RDD持久化到集群内的磁盘，并不完全可靠
 - 检查点机制将RDD写入外部可靠的分布式系统如HDFS
 - 实现层面，写检查点是一个独立的作业，在用户作业结束后运行
- 编程示例
 - 框架示例

- sparkConf指定应用程序的名称、Master等配置信息，创建维护Spark程序信息的SparkContext对象
 - 按照应用逻辑使用操作算子编写DAG，包括RDD的创建、转换、行动等
 - 关闭与spark集群的连接
- 词频统计
- 关系表自然能连接及其优化
- 网页链接排名
- K均值聚类
- 检查点
- 总结
 - 设计思想：数据模型抽象为弹性分布式RDD数据集；计算模型抽象为一系列RDD依赖组成的DAG
 - 体系架构：主从架构
 - 工作原理：通过RDD依赖关系构建DAG，划分Stage，内部pipeline，之间shuffle，区分应用和作业的概念
 - 容错：RDD持久化、RDD Lineage、检查点机制
- 习题
 - 1.与spark相比，MR的局限性
 - 编程框架的表达能力有限，用户编程复杂
 - 单个作业的shuffle阶段的数据需要以阻塞方式传输，磁盘IO开销大、延迟高。
 - 多个作业之间衔接涉及IO开销，应用程序的延迟高
 - 2.说明spark的逻辑计算模型和物理计算模型之间的关系
 - 逻辑计算模型中，DAG从算子的角度描述计算过程；Lineage从RDD数据变换的角度描述计算过程。
 - 物理计算模型源于逻辑计算模型，逻辑模型中的每个算子在物理上可能会有多个并行的实例。
 - 3.什么是RDD Lineage？当Lineage较长时，如何加快故障恢复？
 - Lineage是Driver的SparkContext中维护的记录RDD转换的DAG，RDD Lineage通过读入外部数据源进行RDD创建，经过一系列转换操作，每次生成不同的RDD以供下一次转换操作使用。最后一个RDD经过行动操作进行转换，输出到外部数据源/
 - 某一RDD的部分分区丢失时，可通过Lineage获取数据的上游来源信息来重新计算和恢复。
 - 若Lineage较长，需要在适当的时机设置数据检查点，将中间部分RDD写入可靠的外部分布式文件系统，再在需要的时候进行检查点计算读取，依据检查点的重新计算可以加快恢复进程。

- 4.简述Standalone Client和Cluster两种模式下Spark架构之间的区别，并画出两种模式下的架构图
 - o Client中Driver在客户端，客户端可以看见程序执行过程中的信息；
 - o Cluster中主节点指定一个Worker启动一个DriverWrapper作为驱动器进程，客户端无法看到过程信息。
- 5.Spark如何划分DAG中的Stage？
 - o RDD的依赖关系，窄依赖划分在Stage内，宽依赖则生成新的Stage
- 6.Spark中的应用和作业存在何种关系？
 - o Application应用程序由多个Job作业组成
 - o （逻辑：Application-一个或多个DAG，一个DAG对应一个物理的Job）
- 7.Spark中的Stage和Task存在怎样的联系？
 - o Stage=TaskSet=多个Task
 - o Stage是Job调度的基本单位；Task是executor上的工作单元
- 8.Spark中的Stage内部如何进行数据传输？之间？
 - o 内部pipeline；之间shuffle；
 - o shuffle Write：SMT将输出RDD的记录按照partition函数划分到相应的bucket中，物化到本地磁盘形成ShuffleblockFile
 - o Shuffle Read：SMT或RT根据partition函数读取相应的ShuffleblockFile，存入缓存区并继续后续的计算
- 9.Spark中的RDD持久化和检查点机制存在哪些异同点？
 - o 相同：都提供容错
 - o 不同：
 - RDD是在spark内部某些节点存储多个备份，检查点是将RDD写入HDFS，更可靠
 - RDD持久化可以加快计算速度，并且作为备份快速恢复因故障丢失的数据分区，而检查点机制仅服务于spark的故障恢复
 - RDD持久化发生在计算过程中，而检查点是在作业结束后单独启动一个作业进行
 - 生命周期：RDD持久化在程序结束后清除，检查点在程序结束后依然存在
 - 持久化RDD会保留RDD Lineage，使用检查点会失去RDD Lineage，具体即将下一个RDD的依赖设置为这个检查点本身，而非之前的RDD Lineage
- 10.Spark中的广播变量机制通常用于什么场景？
 - o 节约内存的场景，如小表和大表的自然连接，大表的shuffle开销大，把小表广播出去可以避免大表进行shuffle

5.资源管理系统Yarn

Yarn: Yet Another Resource Negotiator

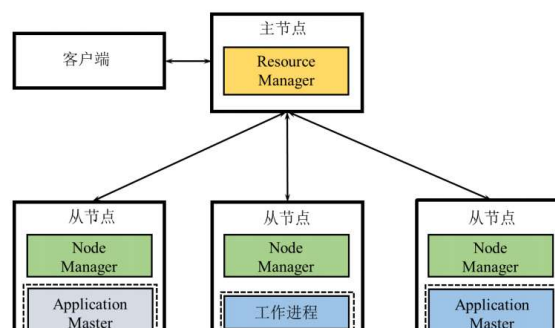
设计思想

- 作业与资源管理
 - o MR1.0的缺陷
 - 资源管理与作业紧密耦合：资源管理不单是MR系统需要的，而是通用的
 - 作业的控制管理高度集中：
 - JT需要维护所有作业的元信息，内存开销大
 - 当同一时刻执行的作业数量增加时，JT与执行这些作业中的任务以及TT之间的通信频率增大，造成JT进程的不稳定
- 平台与架构
 - o 系统
 - 平台：具有提供资源功能的系统
 - 框架：运行在平台上的系统
 - o Yarn应用
 - Yarn管理的粒度是应用
 - 但不一定就是框架中的应用
 - 运行在Yarn这个平台上的框架可以将应用或作业映射为Yarn的应用
 - 比如Yarn中的应用可对应Spark中

• 体系架构

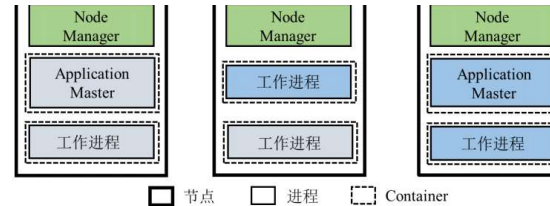
- 架构图

- o 主节点ResourceManager资源管理器
 - 负责整个系统的资源管理和分配
 - 资源管理器ResourceScheduler
 - 分配Container并进行资源调度



分布式系统

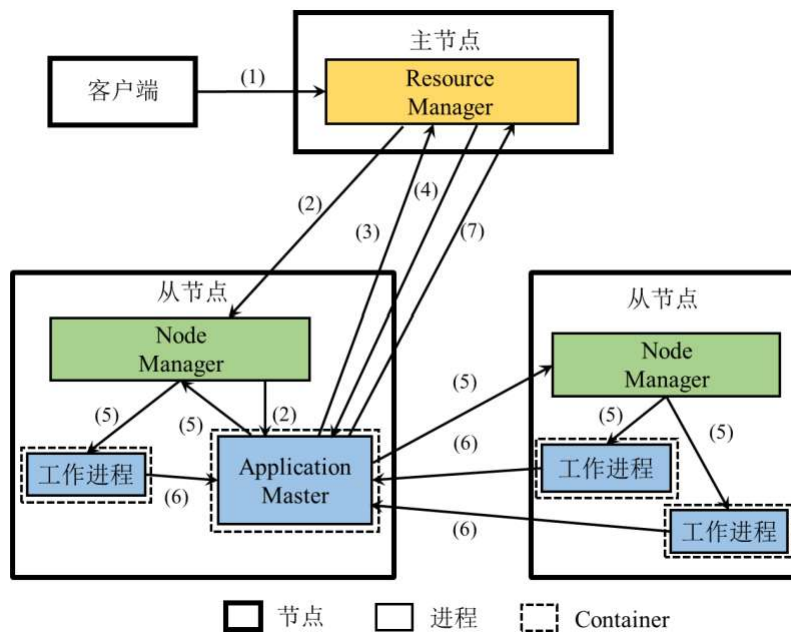
- 资源管理器ResourceScheduler
 - 分配Container并进行资源调度
- 应用程序管理器ApplicationManager
 - 管理系统中运行的所有应用



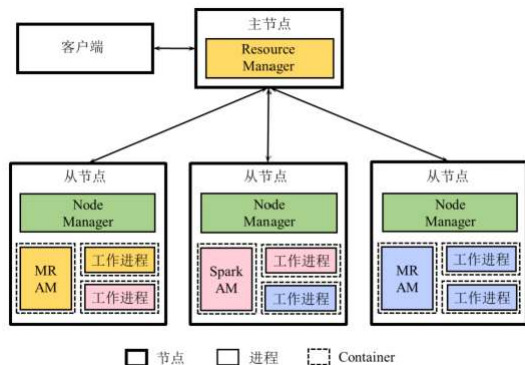
- 应用程序提交
 - 与调度器协商资源以启动ApplicationMaster
 - 监控ApplicationMaster运行状态
- 从节点NodeManager节点管理器
 - 定时向RM汇报本节点的资源使用情况和Container运行状态
 - 接受并处理来自AM的Container启动/停止等各种请求
- ApplicationManager
 - 当用于基于Yarn平台提交一个框架应用，Yarn均启动一个AM用于管理该应用
 - AM与RM协商以获取资源（Container），将获取的资源进一步分配给应用内部的任务
 - AM与NM通信以启动/停止任务，监控所有任务运行状态，并在任务发送故障时重新申请资源来重启任务
- Container
 - 资源的抽象表示，包含CPU、内存等资源，是一个动态资源划分单位
 - 当AM向RM申请资源时，RM向AM返回以Container表示的资源
- 独立出来的资源管理系统，MR2.0作为计算系统负责作业管理

MapReduce 1.0	功能	MapReduce2.0 + Yarn
JobTracker TaskTracker	资源管理	ResourceManager
	作业管理	NodeManager
		ApplicationMaster
Task	执行计算	Container

- 应用程序执行流程

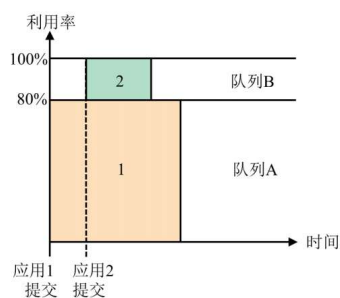


- 用户编写客户端应用程序，向Yarn提交应用程序
 - RM接收和处理来自客户端的请求，尝试为该程序分配第一个container，若分配成功则在Container中启动应用程序的AM
 - AM向RM注册，这样客户端可通过RM查看应用程序的资源使用情况。AM将应用解析为作业并进一步分解为若干任务，并向RM申请启动这些任务的资源
 - RM向申请的AM分配以Container表示的资源，AM申请到后，在多个任务间进行资源分配
 - AM确定分配方案后，与对应的NM通信，在相应的Container中启动工作进行用于执行任务
 - 各个任务向AM汇报自己的状态和进度，以便让AM随时掌握各个任务的运行状态
 - 随着任务执行结束，AM逐步释放所占用的资源，最终向RM注销并关闭自己
- 工作原理
 - 单平台多框架：一个资源管理平台运行多个计算框架



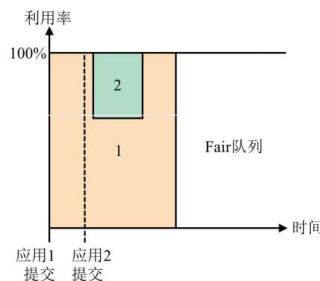
- 平台资源分配

- RM中的调度器维护了一个或多个应用队列，每个队列拥有一定资源，位于同一队列中的应用共享该队列所拥有的资源
- Yarn进行资源分配对象是应用**，用户提交的每个应用会分配到其中一个队列当中，队列决定了该应用能使用的资源上限
- 资源调度实际上是决定如何将资源分配给队列、以及如何分配给队列中应用的过程
- 资源分配策略
 - FIFO
 - 只维护一个队列，该队列拥有集群中所有资源，资源分配是FIFO
 - Capacity
 - Capacity Scheduler维护层级式的队列，队列内部资源分配方式是FIFO



Capacity Scheduler调度方式可以避免某一长时间运行的应用独占集群资源而其它应用得不到运行的情况。

然而，我们也能观察到，在提交应用2之前队列B中的资源处于空闲状态，这造成了集群资源的浪费。



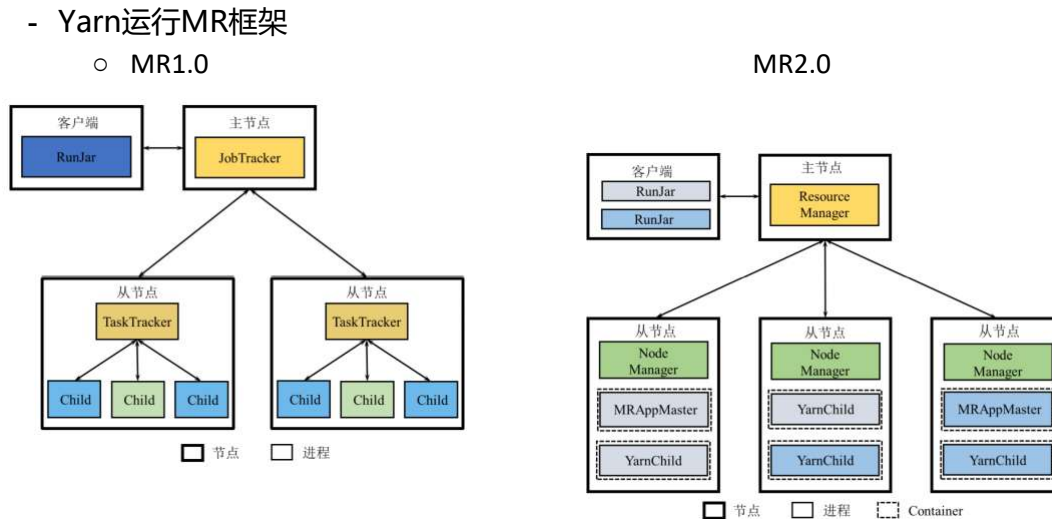
当只有一个应用运行时，这个应用可以独占整个集群。但当其它应用提交到集群时，将空出部分资源给新的应用，最终所有的应用会根据所需使用内存的大小得到分配的资源。

▪ Fair

- Fair Scheduler维护层级式队列，队列逻辑上可看作共享队列，共享资源

• 容错机制

- RM故障
 - 从某一持久化存储系统中恢复状态信息，所有应用将会重新执行
 - 可部署多个RM通过zooKeeper进行协调，从而保证RM的高可用性
- NM故障
 - RM可通过NM的心跳判断是否发生故障；
 - RM会将执行失败的信息告诉AM
 - AM将向RM重新申请资源运行这些任务
 - RM将分配其他节点的Container执行这些任务
 - 若发生故障的NM进行恢复，那么它将向RM重新注册，重置本地的状态信息
- AM故障：重启
- C中任务故障：重启
- 典型示例



	MapReduce 1.0	MapReduce 2.0 + Yarn
资源管理	JobTracker	ResourceManager
应用管理	TaskTracker	NodeManager
任务执行	Child	MRAppMaster
		YarnChild

- yarn运行Spark框架
 - 不适用yarn：Spark的Standalone模式

○ 分类

▪ Yarn Cluster模式:

- 客户端向RM提交一个应用，RM随机选一个NM启动AM进程，Driver运行于该进程中并由该进程负责管理该Spark应用。
- AM根据Yarn分配的资源进行任务调度，并且的CGEB进程执行任务

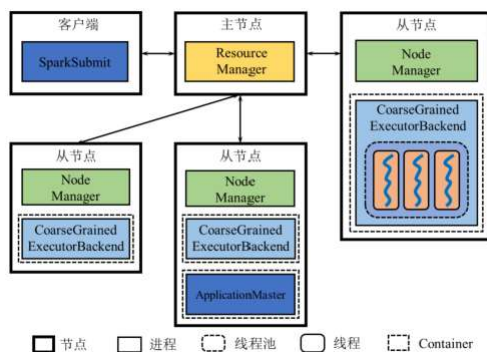
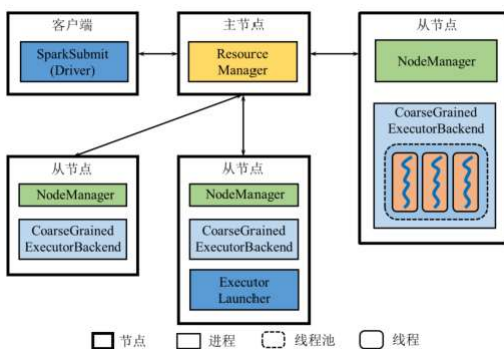


表 5.2 Spark 的 Standalone 与 Yarn 模式对比

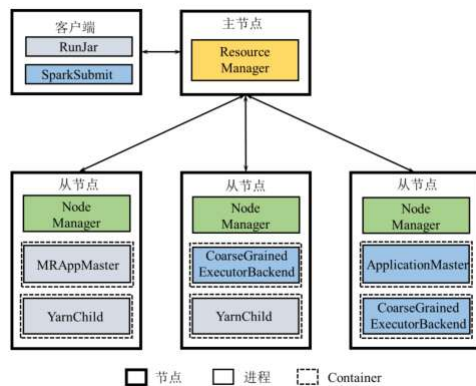
模式	Standalone		Yarn	
	Client	Cluster	Client	Cluster
资源管理	Master		ResourceManager	
	Worker		NodeManager	
应用管理	SparkSubmit	DriverWrapper	SparkSubmit	ApplicationMaster
任务执行	CoarseGrainedExecutorBackend			

▪ Yarn Client模式:

- 客户端向RM提交一个应用，RM随机选取一个NM启动ExecutorLauncher的进程
- EL仅负责向RM申请资源启动CGEB进程，不负责运行Driver，不管理资源也不管理应用
- Driver仍运行在客户端的SparkSubmit进程中。
- 主要用于代码调试，生产下应使用Cluster



- Yarn运行MR和Spark框架



- 总结

- 设计思想

- Yarn将资源管理和作业分离使得作业之间可以相互独立地执行，解决MR资源管理和作业紧密耦合和作业控制管理高度集中带来的问题
- 区分了平台和框架的概念

- 体系架构

- RM和NM的主从架构
- 支持各类应用程序的执行

- 工作原理

- 单平台多框架
- 资源分配策略：FIFO、Capacity、Fair

- 容错机制

- RM/NM故障

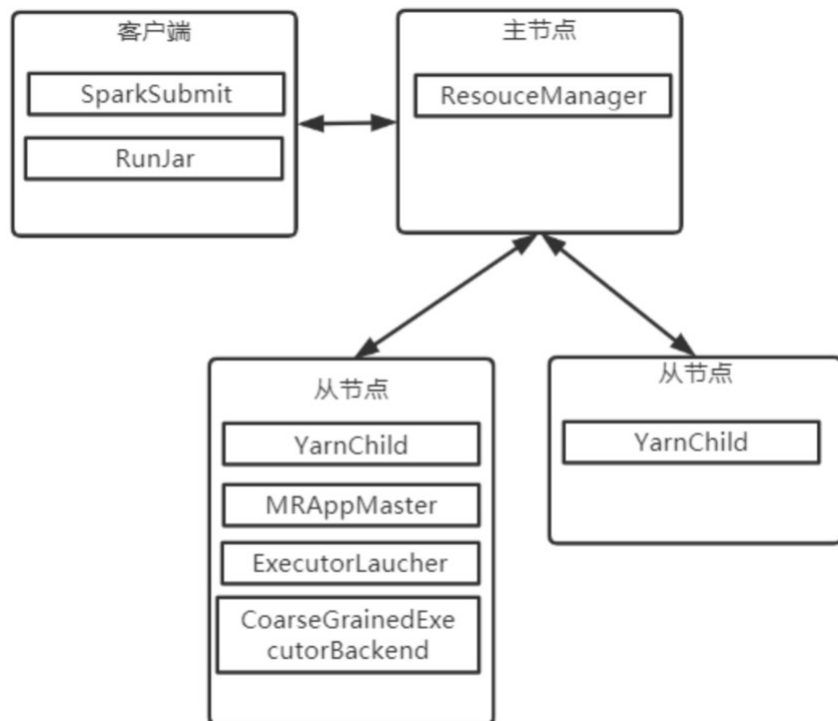
- 资源管理系统按照资源分配时所采用的调度方式可以分为

- 中央式调度：Yarn：可扩展性较弱，无法灵活地为底层的各个框架定制不同的调度策略
- 双层调度：在中央式调度器这一层下添加了若干框架调度器，中央调度器负责向框架调度器提供资源，框架调度器负责向属于该框架的应用分配资源
- 状态共享调度：多个框架调度器，移除了中央调度器，需额外的机制解决冲突。

习题

- 1.对于MR和Spark而言，应用与作业是否存在区别
 - MR应用=作业
 - spark应用可能等于很多作业
- 2.spark架构与Yarn架构在设计思想上有无共同点
 - 主从架构
 - 资源管理和作用管理分离，使作业之间可以相互独立地控制执行
- 3.第二代Hadoop与第一代Hadoop相比所具备的优势是什么
 - 1.0中MR资源管理和作业管理紧密耦合，JT既负责作业管理也负责资源调度；作业管理高度集中于JT，负责管理所有作业的管理，内存开销更大，作业数量增加时，JT与各个作业所在节点的通信增大，造成开销增大，JT进程不稳定。
 - 2.0MR分离两者，RM、NM负责资源管理，MR应用提交后，Yarn启动MRAppMaster管理该应用，负责进行任务的调度，启动YarnChild执行任务。
- 4.Yarn主要包含哪些部件？各个部件分别有什么作用？
 - RM：整个系统的资源调度和分配的紫云管理器
 - NM：负责节点的资源和任务资源的节点管理器
 - AM：每启动一个应用Yarn就会启动一个AM以管理该应用，AM与RM协商获取资源，将资源进一步分配给作业内部
 - Container：资源的抽象表示，RM向AM返回以Container表示的资源
- 5.Yarn中Container内的任务由谁负责启动/停止
 - AM向RM申请获取，任务结束后，AM逐渐释放占用的资源，最终向RM注销并自己关闭。
- 6.简要分析FIFO/Capacity/Fair三种调度器的优缺点
 - FIFO：仅维护一个队列，拥有集群上所有资源，可能会造成一个进程独占资源
 - Capacity：维护一个层级式队列，避免所有资源被一个进程独占，但可能造成一些队列资源空闲
 - Fair：允许队列间资源共享，避免资源浪费，但实现比较复杂，进程之间的切换容易产生较大的代价
- 7.简述AM申请资源的过程
 - AM将作业解析为若干任务，向RM申请启动这些任务的资源

- RM提供Container表示的资源
- AM确定资源在各个任务之间的分配方案
- AM确定分配方案后，与对应的NM通信在相应的Container中启动相应进程执行任务
- 8.NM是否监控Container中任务的运行情况
 - no。AM监控
- 9.AM由谁监控？简述其容错恢复过程
 - RM监控。AM故障，Yarn会重启进程，但不会恢复运行状态，运行状态由框架恢复
- 10.为什么引入Yarn后，MR无法独立运行
 - JT和TT消失，资源管理依靠RM和NN，作业管理依靠MRAM，MR不能本身进行资源和作业管理
- 11.Yarn平台运行MR的方式与Spark的Yarn Cluster和Yarn Client两种模式哪一种更相似，为什么？
 - cluster。启动AM负责应用管理，对应MRAM；而Client的EL只负责分配资源和启动任务，不负责应用管理
- 12.画出Yarn平台同时运行MR和Spark的架构图，其中Spark以Yarn Client模式运行



10.Flink

简介

- 起源：stratosphere项目
- 相关人员：Berlin的。Volker Markl教授、Stephan Ewen博士
- 最初是批处理系统，扩充MR算子，引入流水线方式进行数据传输
- 谷歌发布Dataflow模型后。Flink逐步定位为批流一体化的执行引擎并支持Dataflow模型中定义的批流融合操作

设计思想

以流计算为中心，将有界数据视为无界数据的特例

- 数据模型：DataStream
 - o 一个可以包含重复项的，记录可以是无界的，但不可变的数据集合
 - o DataSet表示有界数据，同样不可变。
 - o 区别：
 - MR：KV；spark：RDD；Storm：Tuple；Flink：DataStream
 - MR和Storm均将数据抽象到记录级别
 - spark和Flink均将数据抽象到记录集合的级别
 - MR和Spark数据有界；Storm和Flink数据无界
- 计算模型
 - o 数据源DataSource：DataStream数据来源
 - o 转换Transformation：系统中的转换逻辑
 - o 数据池DataSink：数据走向，标志DAG结束（Spark中为Action）

表 10.1 Flink 中常见的数据源

操作算子	含义
fromElements(elements)	从相同类型的记录创建 DataStream
fromCollection(collection)	从内存集合创建 DataStream

表 10.2 Flink 中常见的转换

操作算子	含义
map(func)	对 DataStream 中的每个记录均使用 func 进行转换，返回一个新的 DataStream

表 10.1 Flink 中常见的数据源

操作算子	含义
fromElements(elements)	从相同类型的记录创建 DataStream
fromCollection(collection)	从内存集合创建 DataStream
readTextFile(path)	逐行读取文件内容创建 DataStream
socketTextStream(hostname, port)	接收来自套接字的内容创建 DataStream
addSource(customer-source-func)	使用用户自定义的 source func 创建 DataStream

表 10.3 Flink 中常见的数据池

操作算子	含义
print()	将 DataStream 写入标准输出
writeAsText(path)	将 DataStream 以文本格式写入指定路径的文件
writeToSocket(hostname, port, schema)	将 DataStream 作为字节数组写入套接字, 输出格式由 schema 指定
addSink(customer-sink-func)	使用用户自定义的 sink func 作为数据池操作

表 10.2 Flink 中常见的转换

操作算子	含义
map(func)	对 DataStream 中的每个记录均使用 func 进行转换, 返回一个新的 DataStream
filter(func)	过滤出对 DataStream 中的每个记录均使用 func 后返回值为 true 的记录
flatMap(func)	与 map 类似, 但能够将 DataStream 中的每个记录映射为 0 个或多个新的记录
union(otherDataStreams)	若干数据类型相同的 DataStream 取并集得到一个新的 DataStream
connect(otherDataStream)	两个数据类型可能不同的 DataStream 取并集得到一个新的 DataStream
keyBy(key)	以给定的 key 划分 DataStream 以创建 KeyedStream
window(WindowAssigner)	将 KeyedStream 中按键分组的记录根据 WindowAssigner 划分为多个窗口, 返回一个 WindowedStream
windowAll(WindowAssigner)	将 DataStream 中的记录根据 WindowAssigner 划分为多个窗口, 返回一个 AllWindowedStream
reduce(func)	通过使用 func 简化 DataStream 中的记录, 返回一个新的 DataStream
aggregate(func)	对 DataStream 中每个窗口的记录使用 func 聚合为结果记录, 返回一个新的 DataStream
join(otherDataStream)	[K,V1]和[K,V2]分别属于两个 DataStream, 返回一个[K,V1,V2]组成的 JoinedStream
coGroup(otherDataStream)	[K,V1]和[K,V2]分别属于两个 DataStream, 返回一个[K,Iterable<V1>,Iterable<V2>]组成的 CoGroupedStream

- 通常来说Flink系统中的一个应用对应一个DAG，Spark中一个应用对应一个或多个DAG

- 迭代模型

- Flink将迭代部分整体视为一个算子，计算的过程仍然是一个DAG
- 批式迭代中，输入数据是有界的，每轮迭代计算的全部结果一般均作为下一轮迭代计算的输入，直到迭代结束
- 流式迭代中，通常每轮迭代计算的部分结果作为输出向后传递，另一部分结果作为下一轮迭代计算的输入，迭代无限

• 体系架构

- 架构图

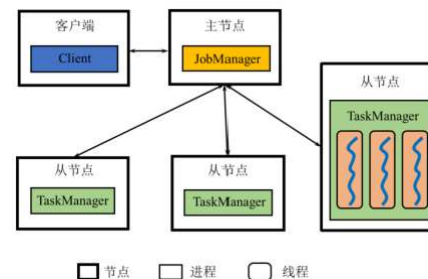
未使用驱动器管理作业，无两C模式。

○ Client客户端

- 将用户编写的DS应用程序翻译为逻辑执行图并优化，将优化后的逻辑执行图提交到JM，系统运行时client进程名为CliFrontend

○ JobManager作业管理器

- 根据逻辑执行图生成物理执行图，负责协调系统的作业执行，包括任务调度、协调检查点和故障恢复等。
- standalone模式下同样负责Flink的资源管理

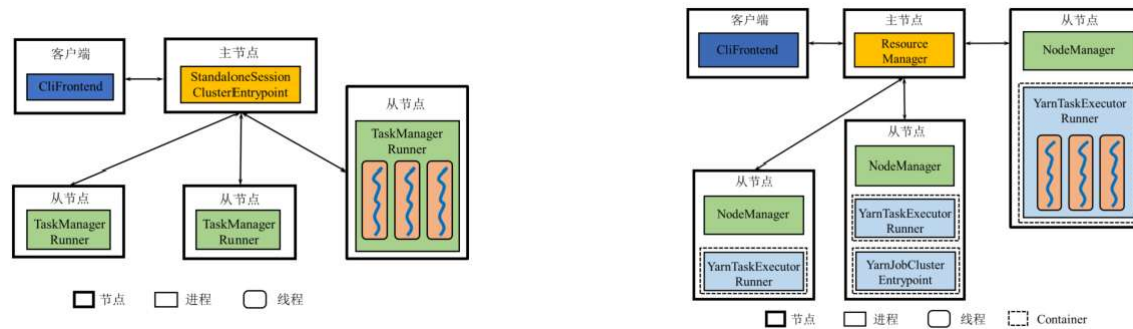


系统运行时进程名为standaloneSessionClusterEntrypoint

TaskManager任务管理器

- 执行JM分配的任务，并且负责读取数据、缓存数据以及与其他TM进行数据传输
- standalone模式下同样负责节点的资源管理，将内存等资源抽象为若干TaskSlot并用于任务执行。

系统运行时进程名为TaskManagerRunner



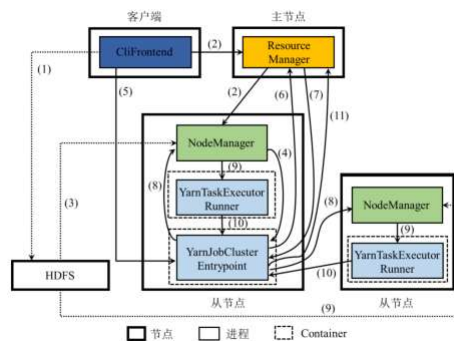
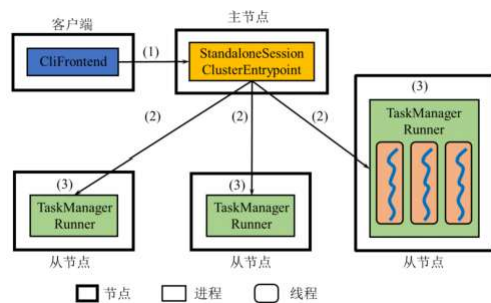
	MapReduce	Storm	Spark	Flink
系统进程	JobTracker	Nimbus	Master	JobManager
	TaskTracker	Supervisor	Worker	TaskManager
工作线程	Child	Worker	CoarseGrained ExecutorBackend	Task
	/	Executor	Task	Task
任务代码	Task	Task		
基础接口	Map/Reduce	Spout/Bolt	RDD API	DataStream API

模式	Standalone	Yarn
资源管理	StandaloneSessionClusterEntrypoint	ResourceManager
作业管理	TaskManagerRunner	NodeManager
任务执行	TaskManagerRunner	YarnJobClusterEntrypoint

- 应用程序执行流程

standalone模式

- 客户端将用户编写的程序解析为逻辑执行图，交给SSCE
- SSCE根据作业描述进行任务分解，确定各个TMR负责执行的任务
- TMR执行各自负责的任务
- 本模式下，用户在客户端提交Flink应用程序时，可以选择
 - Attached：客户端与JM保持连接，可以获取关于应用程序执行的信息
 - Detached：断开连接，无法获取应用程序执行信息



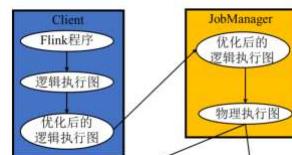
○ Yarn模式

- 用户在客户端提交Flink作业，客户端启动CliFrontend进程，CF将用户编写的程序解析，并将运行Flink系统的jar包以及配置文件上传至HDFS
- CF向RM发送请求，申请启动YarnJobClusterEntrypoint (AM)，RM确定启动节点
- 对应的NM将HDFS中的jar包和配置文件下载至该节点
- NM启动YJCE进程
- CF进程将解析后的作业描述交给YJCE
- YJCE向RM注册，根据作业描述进行任务分解，向RM申请启动这些任务的资源
此时客户端可以通过RM查看Flink应用程序的资源使用情况
- RM以Container形式向申请的YJCE分配资源，YJCE在多个任务间进行资源分配
- YJCE确定资源分配方案后与对应的NM通信
- 若NM节点尚未下载jar包和配置文件，则先从HDFS下载到本地，并在相应的Container中启动相应的YarnTaskExecutorRunner进程执行任务
- 各节点向YJCE汇报状态和进度，让YJCE掌握各个任务的运行状态
- 随着部分任务执行结束，YJCE逐步释放占用的资源，最终向RM注销并自行关闭
- 本模式下，用户同样可选择
 - Attached：CF将与YJCE保持连接，获取应用程序执行西悉尼
 - Detached：断联，无法获取

• 工作原理

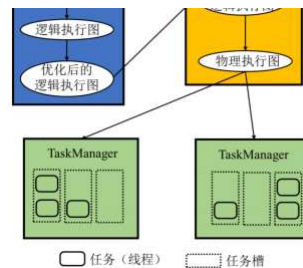
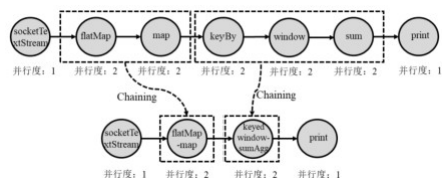
- 逻辑执行图的生成与优化

- 给定用户编写的DS程序，Client将其解析产生逻辑执行图，



- 逻辑执行图的生成与优化

- 给定用户编写的DS程序，Client将其解析产生逻辑执行图，然后使用chaining机制优化
- 类似spark的Stage，算子之间为窄依赖时合并生成一个新算子，避免数据在不同TM之间的非必要传输



- 物理执行图的生成与任务分配

- JM收到Client提交的DAG后，根据算子的并行度转换为物理执行图
- 物理执行图中的一个节点对应一个任务，分配给TM执行
 - 分配原则：根据任务槽TaskSlot容量，尽可能将存在数据传输关系的算子实例放在同一个任务槽，**保持数据传输的本地性**

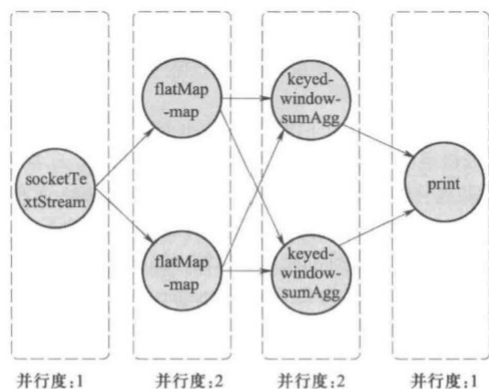
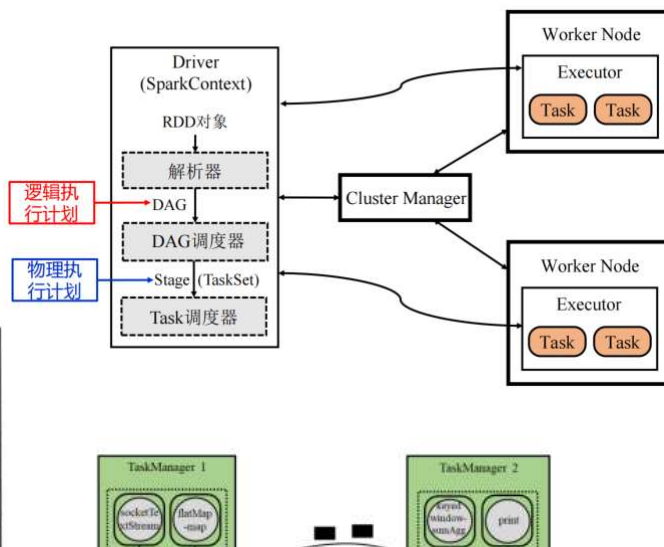
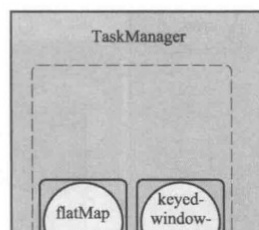
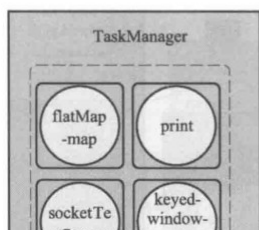


图 10.12 物理执行图



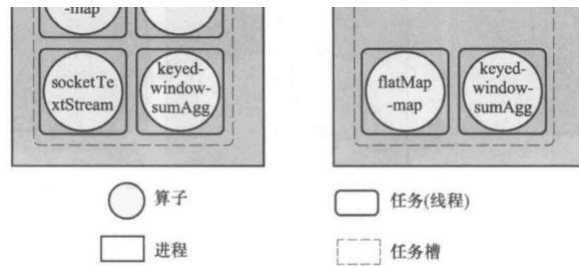
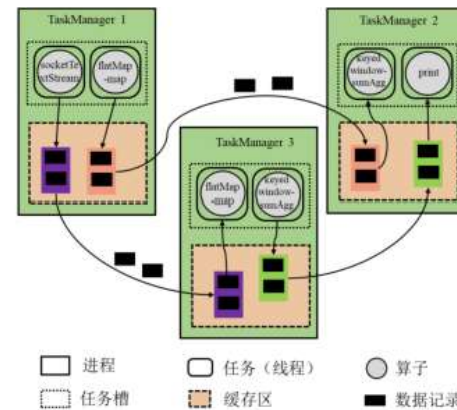


图 10.13 任务分配



- 非迭代任务间的数据传输

o 流水线机制

- 上游的Task将数据存在buffer中，一旦buffer满了或者超时，就向下游Task发送
- 类似storm中的消息传递机制，均为非阻塞式数据传输
 - 传输粒度不同：storm消息传递机制为一次传输一条记录；Flink是传buffer
- 与Spark的流水线区别
 - Flink是不同Task之间数据传输pipeline
 - Spark是内部同一个Task实现多个不同算子间pipeline。不同Task (stage) 之间还是阻塞。
 - DS的pipeline决定其数据是“不完整”的，所以DS无法像RDD那样持久化
Flink也无法像Spark利用RDD Lineage实现故障恢复，所以没有DS Lineage

- 迭代任务内部的数据传输

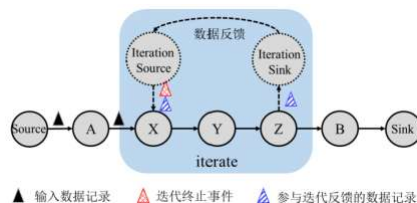
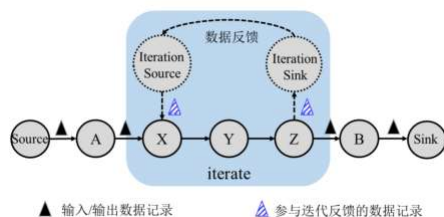
o 迭代算子

- 嵌套在描述计算过程DAG中的一个整体
- 迭代算子内部存在数据反馈的环路

o 数据反馈的实现

- 迭代前端IterationSource和迭代末端IterationSink两类特殊任务
- 两类任务成对处于同一个TaskManager，迭代末端任务的输出可以再次作为迭代前端任务的输入
- 迭代前端下一轮的计算并不依赖于迭代末端在前一轮迭代得到的所有记录，所以可以次采用流水线方式进行数据传输

- 批迭代中，下一轮迭代需要上一次迭代的全部结果，并且直到满足收敛条件停止
迭代前端存在等待迭代末端反馈所有记录的阻塞过程，无法使用流水线传输
需要迭代前端发出特殊的控制事件表示迭代的结束。



• 容错机制

- 状态管理

○ 状态state

- 某窗口需要维护的内容即为状态
- 应该保存在系统里
 - 用户程序编写维护
 - ◆ 一旦算子所在Task故障，内存中的信息就丢失了
 - ◆ 不同数据结构都需要编写相应的保存读取代码，复杂
- 状态：系统定义的特殊的数据结构，用于记录需要保存的算子计算结果

○ 分类

- 有状态算子：具备记忆能力的算子
可以保留已经处理记录的结果，并对后续记录的处理造成影响
eg: sum
- 无状态算子：不具备记忆能力的算子
只考虑当前处理的记录，不会受到已处理的影响，也不会影响后续待处理的记录
eg: map

○ 状态管理与容错

- 算子级别的容错
运行时保存状态，故障时重置状态并继续处理结果尚未保存到状态当中的记录
- DAG级别的容错

如果可以在同一时刻将所有算子的状态保存起来形成检查点，故障时全部重置
但要求所有节点的物理时钟绝对同步——不可能

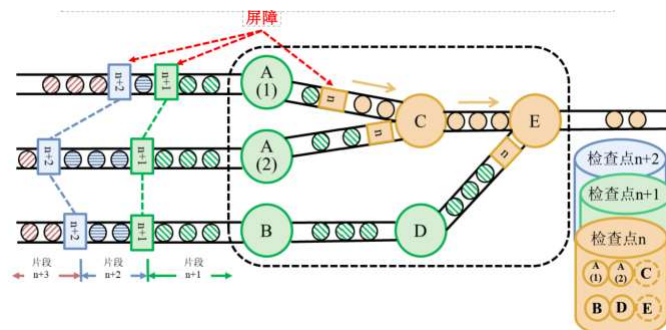
- 非迭代计算过程的容错

o 系统中的记录

- 已经处理完毕的记录/正在处理的记录/尚未处理的记录
- 绝对时钟同步是不存在的，但同一时刻保存所有算子的状态的目的是区分1与23。

o 异步屏障快照ABS

- 借鉴分布式系统中用于保存系统状态的chandy-Lamport算法
- ABS算法
 - 所保存的快照就是检查点
 - 通过在输入数据中注入屏障并异步地保存快照达到和在同一时刻保存所有算子状态到检查点相同的目的。
- 屏障：JM在输入记录中插入屏障，这些屏障与记录一起向下游的计算任务流动
- 屏障对齐：一个任务需收到来自上游任务中所有表示为n的屏障后才将其状态保存起来
- 异步：某一任务标识为n的屏障对齐后，可继续接收属于检查点n+1的数据



状态存储方式	正常运行时	写检查点时
MemoryState Backend	本地内存	JobManager内存
FsStateBackend	本地内存	HDFS
RocksDBState Backend	本地 RocksDB	HDFS

o 故障恢复

- Flink选择最近完整的检查点，将系统中每个算子的状态重置为检查点中保存的状态
- 从数据源重新读取属于屏障n之后的记录
 - 要求数据源具备一定的记忆功能
 - eg: Flink从Kafka中重新读取屏障n对应偏移量之后的记录

- Flink的容错机制能够满足准确一次的容错语义
- 迭代计算过程的容错
 - 仅靠屏障无法将属于n和n+1的记录区分
 - 根据Chandy-Lamport算法, 反馈环路中的所有记录需要以日志形式保存
 - 故障发生后系统需
 - 根据最近完整检查点重置各个算子状态
 - 重新读取n之后的记录以及日志记录
- 仅支持DataStream接口编写的程序, DataSet的无法支持容错, 故障需重新执行整个程序
- 编程示例
- 总结
 - 设计思想: DS作为数据模型, 有内置的迭代算子
 - 体系架构: 主从架构
 - 工作原理: 流水线
 - 容错机制

习题

- 1.Flink使用何种数据模型表示数据? 与MR、Spark、Storm的区别?
 - DataStream。无界
- 2.Standalone模式下, Flink架构包含哪些部件? 各个部件分别有何功能?
 - Client: 将用户编写的DS程序翻译并优化为逻辑执行图, 提交到JM, 运行时叫CliFrontend
 - JobManager: 根据逻辑执行图转化为物理执行图, 负责协调作业的物理执行, 也负责资源管理, 运行时叫StandaloneSessionClusterEntrypoint
 - TaskManager: 执行JM分配的任务, 负责读取数据、缓存数据、和其他节点的数据传输等, 也负责节点的资源管理, 运行时叫TaskManagerRunner
- 3.Standalone模式下, Flink中的JMTM与Spark中的Master和Worker在功能上是否完全一致?
 - no。Spark的Master和Worker都是资源管理, 作业交由Client的Driver管理。而Flink的JM和TM还负责作业管理, JM类似Driver, TM类似Executor

- 4.Standalone模式下，同一个TM可能同时执行不同应用程序的任务，某种程度上会存在应用程序之间的相互干扰，引入Yarn后是如何解决该问题的？
 - o 没引入的时候问题还是资源管理和作业管理未相互分离。引入Yarn后分了，就没了
- 5.Flink中的逻辑执行图和物理执行图分别如何产生？与Spark的区别？
 - o Client解析并优化应用程序为DAG，交给JM，JM转化为物理执行图
 - o Spark里Driver首先解析应用程序为DAG，然后转化根据依赖关系进行Stage的划分。
- 6.Flink中的非迭代任务间如何进行数据传输？与Spark和Storm有何区别？
 - o pipeline传输，一次传一个buffer
 - o Spark是Shuffle；Storm是一次一条记录
- 7.简述异步屏障快照的工作过程
 - o 在输入数据中注入屏障并异步地保持快照，达到和在同一时刻保存所有算子状态到检查点相同的目的。
 - o 数据不断流入系统，当一个任务收到所有来自标识为n的屏障后将其状态保存，所有算子保存的状态形成检查点n。如果此时发生故障，则将所有算子对应的任务还原为检查点对应的状态
- 8.Flink中的迭代算子如何实现数据反馈？
 - o 迭代前端ISource和迭代末端ISink，成对处于同一个TM，迭代末端任务的输出部分作为下一次迭代前端的输入，部分作为输出向后传递
- 9.如果Flink系统不进行状态管理，将造成哪些缺陷？
 - o 没容错。
 - o 状态是Flink提供的用于保存算子的计算结果的特殊数据结构。算子的状态相当于检查点，系统可以根据前一段时刻保存的算子的状态对于出现故障的系统进行恢复。否则只能完全从头开始计算，这对用户来说是无法实现的，因为Flink到来的是数据流。
- 10.MemoryStateBackend和FsStateBackend两种状态存储方式中，哪一种更适用于实际生产环境？为什么？
 - o 后者。
 - o 前者是在保存检查点时把状态发送给JM并且存储至其内存，而后者是保存检查点时将状态存

储于HDFS等文件系统。

- 后者更适合实际生产环境、JM的内存有限，可能无法保存全部信息；JM故障也可能导致信息丢失，而HDFS本身具备容错。
- 11.Flink为何无法像Spark那样利用Lineage进行故障恢复？
 - Lineage可进行的是基于静态数据的故障恢复；Flink是动态的，pipeline的传输方式，没有类似RDD的数据持久化。发生故障时每个节点上都有可能对数据进行计算，无法知道故障时每个节点的准确状态，因此无法进行故障恢复。

未命名

2026年6月28日

19:41